

Федеральное государственное образовательное бюджетное
учреждение высшего образования
**«Финансовый университет
Правительства Российской Федерации»
(Финансовый университет)**

Факультет информационных технологий и анализа больших данных
Кафедра информационных технологий

Выпускная квалификационная работа

На тему «Разработка геоинформационной системы управления аэропортом»

Направление подготовки 09.03.03 «Прикладная информатика»

Профиль программы ИТ-сервисы и технологии обработки данных в
экономике и финансах

Выполнил студент учебной группы
ПИ20-1В

Зайцев Никита Валерьевич

Руководитель д-р экон. наук, профессор

Демин И.С.

**ВКР соответствует предъявленным
требованиям**

Заведующий кафедрой
информационных технологий, к.т.н.,
доцент

_____ Д. А. Петросов

«_____» _____ 2025 г.

Москва – 2025 г.

Содержание

Введение.....	3
1. Теоретическая часть.....	5
1.1. Постановка задачи	5
1.2. Анализ предметной области	9
2. Проектирование приложения.....	23
2.1. Выбор и обоснование средств разработки	23
2.1.1. Общая архитектура приложения	23
2.1.2 Выбор языка и стека разработки	25
2.2 Описание интерфейса приложения.....	38
3. Разработка приложения	42
3.1. Процесс разработки приложения	42
3.1.1 Backend часть приложения.....	42
3.1.2 Frontend часть приложения	51
3.2. Проверка корректности работы.....	59
3.2.1. Общие сведения.....	59
3.2.2. Тестирование backend части приложения	61
3.2.3. Тестирование frontend части приложения	63
Заключение	65
Список использованных источников	67
Приложение А	71

Введение

В современном мире, с каждым годом все сильнее ощущаются тенденции стремления к многополярности. Смена условий мироустройства, когда старые поставщики услуг становятся ненадежными и могут ограничить к ним доступ для каждого мирового государства, все чаще встают вопросы, касающиеся его технологической независимости. Исходя из этого, страны начинают задумываться о замещении импортных технологий на собственные отечественные разработки. Россия так же входит в число этих стран. С 2014 года наблюдается тенденция стран Запада по отношению к России ограничивать доступ к своим технологиям, исходя из политических амбиций. В связи с этим Правительство РФ приняло решение о начале процесса импортозамещения для обеспечения собственной технологической независимости. Для этого было выпущено Постановление от 4 августа 2015 г. № 785, которое предписывает создание Правительственной комиссии по импортозамещению.

Важность импортозамещения заключается в нескольких причинах:

- Развитии собственного производственного потенциала. После распада СССР Россия утратила значительную часть производственных мощностей. Импортозамещение позволяет создать собственную производственную базу, способную поставлять аналоги зарубежных товаров.
- Протекция внешних и внутренних экономических интересов. Политика импортозамещения защищает интересы страны и стимулирует развитие национальных компаний.
- Минимизация геополитических и финансовых рисков.
- Создание новых рабочих мест, в том числе высокотехнологичных.
- Увеличение доходов и благосостояния населения.
- Стимулирование экономического роста.

- Обеспечение экономической и продовольственной безопасности государства.

Таким образом, импортозамещение сегодня играет значимую роль на рынке отечественного ПО.

Если переходить от общих понятий к конкретным, то ярким примером может послужить импортозамещение в рамках критической и стратегической инфраструктуры, которыми являются аэропорты на территории РФ. В современном аэропорте используется больше количество программного обеспечения касательно вопросов автоматизаций процессов, преимущественно произведенных в странах Запада. Это может являться серьезной уязвимостью в случае ухудшения политической конъюнктуры, так как ПО может быть завязано на облачные модули, которые могут быть дистанционно выключены. В этих рамках компания ООО «Код Открытия» занимается вопросом замещения в аэропортах иностранного программного обеспечения на отечественное, обеспечивающее надежное функционирование в рамках мировых кризисов.

1. Теоретическая часть

1.1. Постановка задачи

Программное обеспечение для аэропортов — это широкий спектр технологий и систем, которые предназначены для оптимизации и управления различными процессами, происходящими в аэропортах. Оно охватывает множество аспектов, включая управление воздушным движением, обработку пассажиров, управление грузами, безопасность и многие другие функции. Рассмотрим основные типы программного обеспечения, используемого в аэропортах:

1. Системы управления аэропортами (Airport Management Systems - AMS). Эти системы помогают управлять всеми аспектами функционирования аэропорта, включая планирование, координацию операций и мониторинг. Они обеспечивают централизованное управление ресурсами аэропорта и могут включать модули для управления взлетно-посадочными полосами, терминалами и парковками.
2. Системы обработки пассажиров (Passenger Handling Systems - PHS). Эти системы обеспечивают обработку пассажиров на всех этапах их путешествия. Они могут включать регистрацию, управление багажом, контроль безопасности и процедуры посадки. Использование таких систем позволяет улучшить клиентский опыт и сократить время ожидания.
3. Системы управления воздушным движением (Air Traffic Management Systems - ATMS) Эти системы отвечают за координацию движения воздушных судов в зоне аэропорта и в его окрестностях. Они обеспечивают безопасное и эффективное движение самолетов при взлете, посадке и на рулежных дорожках.
4. Системы управления грузами (Cargo Management Systems - CMS). Эти системы помогают эффективно управлять грузовыми операциями, включая принятие, обработку и доставку грузов. Они позволяют

отслеживать перемещение грузов в реальном времени и оптимизировать логистические процессы.

5. Системы обеспечения безопасности (Security Management Systems - SMS). Программное обеспечение для обеспечения безопасности аэропортов включает системы видеонаблюдения, контроля доступа и управления инцидентами. Эти системы помогают обеспечивать безопасность пассажиров и сотрудников, а также предотвращать угрозы.
6. Системы управления пассажирским потоком (Passenger Flow Management Systems). Эти системы анализируют данные о пассажирских потоках и помогают оптимизировать расстановку сотрудников, обработку багажа и организацию пространства в терминалах.
7. Системы информации для пассажиров (Passenger Information Systems - PIS). Эти системы предоставляют пассажирам актуальную информацию о рейсах, терминалах, воротах, задержках и других аспектах. Они могут включать информационные табло, мобильные приложения и интерактивные киоски.
8. Аналитические платформы. Современные аэропорты используют аналитические инструменты для обработки больших объемов данных о пассажирах, грузах и операциях. Эти системы помогают в принятии обоснованных решений, прогнозировании трафика и выявлении тенденций.

Внедрение современного программного обеспечения в аэропортах позволяет значительно повысить эффективность операций, улучшить качество обслуживания пассажиров и обеспечить высокий уровень безопасности. Постоянное развитие технологий также открывает новые возможности для автоматизации процессов и повышения эффективности работы аэропортов.

Тема ВКР касается систем управления аэропортом, поэтому, рассмотрим ее подробнее. Стоит начать с того, зачем России в принципе нужно разрабатывать свои системы управления аэропортом.

Разработка собственных систем управления аэропортами в России имеет несколько ключевых причин, важнейшие из которых описаны ниже:

Национальная безопасность - разработка собственных решений позволяет России снизить зависимость от иностранных технологий и систем, что важно с точки зрения национальной безопасности. В условиях геополитической нестабильности и санкций важно иметь контроль над критически важной инфраструктурой, такой как управление аэропортами.

Технологическая независимость - создание собственных систем управления аэропортами способствует развитию отечественных технологий, снижая зависимость от западных производителей. Это может привести к созданию новых рабочих мест и укреплению высоких технологий в стране.

Экономические преимущества - инвестиции в разработку собственных систем открывают новые возможности для роста и модернизации отрасли. Эти системы могут быть адаптированы к специфическим потребностям российских аэропортов, что может повысить эффективность операций и снизить затраты.

Улучшение качества обслуживания - российские аэропорты могут разработать решения, которые будут лучше учитывать местные особенности и потребности пассажиров. Это позволит оптимизировать процессы, такие как регистрация, досмотр и посадка, улучшая общий опыт путешественников.

Инновации и адаптация - собственные системы обеспечивают больший уровень гибкости и адаптивности. Это позволяет быстро реагировать на изменяющиеся условия рынка, внедрять инновации и улучшать процессы в соответствии с актуальными требованиями.

Поддержка локального производства - разработка собственных систем может стимулировать локальное производство оборудования и программного

обеспечения, что позволит создать новые цепочки поставок и поддержать малый и средний бизнес в области технологий.

Соблюдение стандартов и регуляций - создание национальных систем управления аэропортами позволяет учитывать особенности российских стандартов и нормативов. Это может помочь в интеграции с другими системами и обеспечении более высокой безопасности.

Устойчивое развитие и экология - на основе местных технологий можно интегрировать программные решения, которые будут способствовать более экологически чистым и устойчивым практикам в управлении аэропортами, таким как оптимизация воздуходувок и управление энергопотреблением.

Таким образом, разработка систем управления аэропортами в России — важный шаг к обеспечению технологической автономии, улучшению качества услуг, а также укреплению роли страны на международной арене. Это не только увеличит конкурентоспособность российских аэропортов, но и поддержит стратегический рост всей отрасли.

1.2. Анализ предметной области

Крупнейшие поставщики систем управления аэропортами (Airport Management Systems, AMS) играют ключевую роль в индустрии гражданской авиации, обеспечивая технологии и решения, которые позволяют оптимизировать операции аэропортов по всему миру. Ниже представлен более подробный обзор основных вендоров на этом рынке:

SITA (Société Internationale de Télécommunications Aéronautiques) является одним из ведущих поставщиков IT-решений для авиакомпаний и аэропортов. Эта компания была основана в 1949 году и предложила множество инноваций в области авиаперевозок. Основной продукт — SITA Airport Management, это решение для управления аэропортом, обеспечивающее контроль за движением воздушных судов, управление ресурсами и потоками пассажиров.

Amadeus IT Group — еще один крупный игрок на рынке, известный своими системами резервирования и туристическими решениями. Компания активно развивает свои решения для управления аэропортами. Amadeus Airport Common Use Service (ACUS) — решение для общего использования терминалов, которое позволяет нескольким авиакомпаниям эффективно работать на одной стойке регистрации.

Honeywell предлагает широкий спектр технологических решений в нескольких областях, включая авиацию. Honeywell Airport Solutions включает в себя решения для управления движением самолетов, управления терминалами и системами безопасности.

Siemens предоставляет решения в области автоматизации и цифровизации, включая решения для инфраструктуры аэропортов. Siemens Airport Management System — платформа для управления воздушным движением, ресурсами и безопасностью на территории аэропорта.

Решения для автоматизации процессов и управления энергией.

AviBit фокусируется на разработке специализированных решений для управления аэропортами и обработкой данных.

ARINC (подразделение Rockwell Collins), ныне часть Rockwell Collins, предлагает решения для управления информационными технологиями в авиации.

Veovo разрабатывает современное программное обеспечение для прогнозирования и оптимизации операций в аэропортах.

Каждый из этих поставщиков предлагает уникальные решения, адаптированные для разных потребностей и размеров аэропортов. Рост технологий, таких как искусственный интеллект и IoT, продолжает трансформировать рынок систем управления аэропортами, делая его более гибким и эффективным. Поставщики активно работают над интеграцией своих технологий для обеспечения совместимости и улучшения взаимодействия между различными системами в аэропортах.

Задача замещения всех вышеописанных продуктов – непростая, но компания ООО «Код Открытия» занимается решением замещения этих продуктов. Рассмотрим подробнее данную предметную область.

ООО «Код открытия» — молодая, амбициозная и перспективная команда, разрабатывающая новые решения на мировом рынке ПО с применением искусственного интеллекта, компьютерного зрения и больших данных. С момента своего основания компания стала одним из ведущих поставщиков IT-услуг, предлагая клиентам высококачественные решения для различных отраслей бизнеса.

Команда состоит из опытных специалистов, обладающих глубокими знаниями в области информационных технологий. Компания стремится к постоянному развитию и совершенствованию своих продуктов, чтобы всегда оставаться на шаг впереди наших конкурентов.

Компания создает современные решения для управления аэропортом, в том числе систему управления ресурсами, цифровой двойник аэропорта и аэропортовые базы данных.

У компании 4 основных направления работы:

1. Экосистема аэропорта Шереметьево;

2. Информационные системы в сфере туризма;
3. Центр компетенций 1С;
4. Внедрение и поддержка информационных систем управления предприятием;

Цифровая экосистема Международного аэропорта Шереметьево позволяет в автоматическом режиме управлять аэропортовым производством на основе обработки больших объёмов данных, что помогает оптимизировать затраты и повысить эффективность работы. Экосистема делится на 2 основные части:

Системы управления процессами аэропорта

- Центральную аэропортовую базу данных «Синхрон». Это основной инструмент управления операционной деятельностью аэропорта.
- Интерактивная многоуровневая карта (ГИС) даёт возможность наглядно представлять взаимное расположение различных объектов (земельных участков, коммуникаций, размещения объектов и т. д.). Привязка передвижений к задачам в ГИС позволяет прописывать пространственные и функциональные связи между объектами, моделировать их поведение.
- Автоматизированную систему управления ресурсами с применением графического интерфейса (RMS).
- Система информирования - система визуального информирования пассажиров о состоянии рейсов на основе по-code позволяет оперативно и доступно доносить необходимую информацию до каждого пассажира на информационных экранах.

Базовую схему взаимодействия системы управления процессами аэропорта можно посмотреть в Приложении А.

Сервисы моделирования и мониторинга:

- Систему совместного принятия решений с авиакомпаниями (A-CDM).

- Цифровой двойник — систему долгосрочного и краткосрочного моделирования, анализа и оптимизации деятельности аэропорта с применением методов искусственного интеллекта, в том числе нейросетей.

Например, цифровой двойник ежедневно отслеживает количество посетителей в аэропорту, что помогает руководству точно определить необходимое количество персонала для работы с багажом, билетами, обслуживания воздушных судов и уборки.

Информационные системы в сфере туризма - Компания «Код Открытия» применяет уникальный подход к развитию туристического бизнеса, объединяя два ключевых направления:

- Прямые продажи туров (B2C) – компания создаёт и развиваем прямые продажи туров, чтобы клиенты могли легко и удобно выбирать наилучшие туристические предложения. Цель — сделать процесс планирования отдыха максимально простым и доступным для каждого.
- Автоматизация процессов. Компания внедряет автоматизацию, чтобы сделать сложные процессы бронирования более прозрачными, быстрыми и удобными для клиентов и менеджеров. Это позволяет эффективно управлять разнообразным портфелем продуктов и предоставлять разным категориям клиентов подходящие комплексные решения.

Например, компания запустила обновлённый веб-сайт туроператора «Библио-Глобус», который обеспечивает удобство и скорость подбора и бронирования туров, отелей и билетов. Благодаря интуитивно понятному интерфейсу заказ тура занимает всего несколько минут, а гибкая система оплаты позволяет выбрать комфортный способ оплаты и улететь в отпуск без лишних проблем.

Центр компетенций 1С открыт на базе подразделения компании по поддержке и внедрению ИТ-систем управления предприятием. Он реализует проекты на базе решений:

- 1С: Управление холдингом
- 1С: Бюджетирование
- 1С: Казначейство
- 1С: Бухгалтерия
- 1С: ЗУП
- 1С: Документооборот

Также функциональные области:

- Управление на уровне группы компаний и холдинга
- Управление финансами (бюджетирование, казначейство)
- Бухгалтерский и налоговый учёт
- Управление продажами
- Управление логистикой и складом
- Управление персоналом, расчёт зарплаты
- Управление документооборотом.

Внедрение и поддержка информационных систем управления предприятием - задачи данного направления входят:

Поддержка и развитие систем управления предприятием:

- Системы электронного документооборота на базе HCL Lotus (ex IBM Lotus Notes);
- Системы календарно-сетевого планирования на базе MS Project Server;
- Системы учета и отчетности, зарплаты и управления персоналом на базе 1С;
- Системы учета и отчетности, управления персоналом и расчета заработной платы на базе SAP ERP;
- Системы бюджетирования и аналитической отчетности на базе SAP Business Planning and Consolidation (BPC), Business Intelligence (BO, BW on Hana);
- Системы консолидации финансовой и управленческой отчетности на базе SAP Financial Consolidation (FC);

- Системы Налогового мониторинга на базе SAP S/4HANA, витрины данных SAP Business Objects.
- Бизнес-анализ и формирование требований к ПО;
- Проектирование архитектуры ПО;
- Согласование архитектуры, требований;
- Оценка трудоемкости запросов на изменение;
- Разработка программного обеспечения;
- Исправление ошибок и выполнение необходимых доработок для обеспечения бесперебойной работы ПО;
- Тестирование запросов на изменение;
- Разработка и актуализация эксплуатационной и пользовательской документации;
- Решение инцидентов пользователей, связанных с ошибками функционирования ПО;
- Выполнение запросов на обслуживание, выгрузка/загрузка данных;
- Определение перечня операций и полномочий для ведения матриц ролей и полномочий;
- Проведение обучения пользователей работе с ПО;
- Мониторинг и диагностика ПО;
- Настройка производительности ПО;
- Техническое обслуживание ПО;
- Проведение регламентных работ СУБД;
- Создание копий ПО;

В условиях сложной экономической и политической ситуации в мире для каждой мировой державы актуальным становится вопрос об импортозамещении для обеспечения собственной независимости. Это важная черта, которая позволяет стране находиться в независимом положении от других технологических держав.

Компания ООО «Код Открытия» разрабатывает программно-аппаратные комплексы для обмена данными и создаёт современные решения для управления аэропортом с использованием искусственного интеллекта, компьютерного зрения и больших данных. Среди основных продуктов — система управления ресурсами, цифровой двойник аэропорта и аэропортовые базы данных.

Перейдем к конкретному разделу управления систем управления аэропортом — геоинформационной системе управления. Основным эксплуатантом системы на примере аэропорта Шереметьево является Дирекция Аэродромная, рассмотрим ее и сам аэропорт Шереметьево подробнее.

Аэропорт Шереметьево, расположенный в 29 километрах от центра Москвы, является одним из ведущих транспортных узлов России. В контексте систем управления аэропортом он использует современные технологии и решения для обеспечения эффективной и безопасной обработки пассажиров и грузов. Рассмотрим ключевые аспекты систем управления аэропортом Шереметьево.

Автоматизация процессов - аэропорт активно внедряет автоматизированные системы для снижения времени ожидания пассажиров и оптимизации работы на всех уровнях:

Чекинг и контроль безопасности - используются самоустанавливающиеся киоски для регистрации и автоматизированные системы контроля безопасности, которые помогают ускорить процесс прохождения пассажиров.

Системы отслеживания багажа: Современные решения по отслеживанию и учету багажа позволяют минимизировать риски его утери и ускоряют процесс доставки.

Системы управления движением - эффективное управление воздушным движением является критически важным аспектом работы аэропорта:

Системы управления воздушным движением (АТС) отвечают за координацию взлетов и посадок, а также за управление движением самолетов

на земле. Шереметьево применяет современные радарные и диспетчерские технологии для обеспечения безопасности.

Управление наземным движением: Специальные системы контролируют перемещение наземного транспорта, обеспечивая безопасность на перронах и рулежных дорожках.

Информационные системы - в аэропорту значительное внимание уделяется информационным системам, которые обеспечивают прозрачность и доступ к актуальной информации:

Информационные табло - на всей территории аэропорта установлены электронные табло, предоставляющие актуальную информацию о рейсах, включая время вылета и прибытия.

Мобильные приложения и веб-сайты - аэропорт имеет современные цифровые платформы, которые позволяют пассажирам получать информацию о рейсах, возможностях трансфера и других услугах.

Управление потоком пассажиров - для оптимизации пассажиропотока используются специальные системы:

Системы анализа и прогнозирования - аэропорт применяет технологии для прогнозирования пассажиропотока, что позволяет заранее оценить загруженность и подбирать необходимый персонал.

Организация очередей - используются системы для управления очередями на контроль безопасности и регистрации, что минимизирует время ожидания.

Интеграция с другими транспортными системами - Шереметьево эффективно интегрировано с другими транспортными системами, включая авиационные, железнодорожные и автомобильные:

Трансферные услуги - Предоставление информации о трансферах и возможностях перемещения между терминалами, а также контроль работы общественного транспорта и такси.

Системы логистики грузовых перевозок - эффективные решения для обработки и хранения грузов внутри аэропорта, что содействует развитию грузопотока.

Аэропорт Шереметьево остаётся на передовом уровне по внедрению современных систем управления, что способствует его выдающимся показателям в качестве транспортного узла. Постоянное улучшение технологий и процессов позволяет ему оставаться конкурентоспособным и обеспечивать высокий уровень безопасности и комфорта для пассажиров и авиакомпаний.

Дирекция аэродромная аэропорта Шереметьево — это важное подразделение, ответственное за управление всеми аспектами аэродромной деятельности. Этот аэропорт является одним из крупнейших и самых современных в России и обслуживает как внутренние, так и международные рейсы. Дирекция играет ключевую роль в обеспечении безопасности, эффективности и удобства всех операций, связанных с использованием аэродрома.

Основные функции дирекции аэродромной Шереметьево

- Управление движением воздушных судов: Дирекция контролирует все аспекты движения самолетов как на земле, так и в воздухе, обеспечивая их безопасное и оптимальное перемещение.
- Обеспечение безопасности: важной задачей дирекции является соблюдение всех норм и стандартов безопасности, включая меры по предотвращению авиационных происшествий и обеспечение быстрого реагирования в экстренных ситуациях.
- Планирование и развитие: Дирекция отвечает за разработку стратегий и планов по развитию аэродромной инфраструктуры, включая взлетно-посадочные полосы, перроны, терминалы и служебные здания.
- Координация с другими службами: Дирекция взаимодействует с различными государственными и коммерческими учреждениями,

включая органы надзора, авиакомпании и службы безопасности, что необходимо для обеспечения эффективной работы аэропорта.

- Эксплуатация и техническое обслуживание: обеспечение работоспособности всей аэродромной инфраструктуры, включая поддержание в хорошем состоянии взлетно-посадочных полос, рулежных дорожек и других объектов.

Дирекция может состоять из различных отделов, каждый из которых отвечает за конкретные аспекты аэродромной деятельности:

- Отдел планирования: занимается разработкой проектов по модернизации и оптимизации инфраструктуры.
- Отдел контроля движений: обеспечивает контроль за текущими операциями на аэродроме, включая диспетчеризацию.
- Отдел безопасности: осуществляет мониторинг и управление всеми вопросами безопасности на аэродроме.
- Технический отдел: Ответственен за обслуживание и ремонт оборудования и объектов.

Нормативно-правовая база:

Дирекция действует в соответствии с российскими авиационными нормами и международными стандартами, такими как требования Международной организации гражданской авиации (ИКАО) и Европейского агентства по авиационной безопасности (EASA).

Современные информационные технологии, включая системы управления аэропортами (Airport Management Systems) и геоинформационные системы (ГИС), играют значительную роль в работе дирекции. Они позволяют улучшить процессы, повысить уровень безопасности и качество обслуживания.

Дирекция аэродромная аэропорта Шереметьево выполняет ключевые функции, обеспечивающие безопасное и эффективное функционирование самого большого аэропорта России. Эти обязанности включают как

операционное управление, так и стратегическое развитие, что делает эту дирекцию одним из важнейших звеньев в авиационной отрасли страны.

Исходя из этого, рассмотрим предмет разработки подробнее:

Геоинформационная система управления аэропортом (ГИС УА) — это комплекс программных и аппаратных средств, предназначенный для сбора, хранения, анализа и визуализации пространственных данных, связанных с операциями аэропорта. Такие системы играют ключевую роль в повышении эффективности управления, безопасности и качества обслуживания в аэропортах. Рассмотрим их основные компоненты, функции и преимущества.

Основные компоненты ГИС управления аэропортом:

- База данных пространственной информации: хранит информацию о территории аэропорта, включая распоряжения, карты, схемы и другие пространственные данные.
- Интерфейс пользователя: обеспечивает доступ к информации и инструментам анализа через графический интерфейс, часто с использованием картографических визуализаций.
- Инструменты анализа: позволяют обрабатывать и визуализировать данные для поддержки принятия решений, включая инструменты для пространственного анализа, моделирования и прогнозирования.
- Системы сбора данных: позволяют видеть состояние объектов и процессов в аэропорту. Для автоматического сбора данных используются датчики на различной технике, например датчики Omnicom, установленные на наземную технику.
- Интеграция с другими системами: ГИС может быть интегрирована с другими системами управления, такими как системы управления движением на земле, безопасности, терминалами и так далее.

Функции ГИС управления аэропортом:

- Планирование и проектирование: помогает в разработке новых объектов, анализе нагрузки, оценке пространственных ограничений и оптимизации размещения инфраструктуры.
- Мониторинг активов: позволяет отслеживать расположение и состояние оборудования, транспорта и других объектов, что способствует более своевременному обслуживанию и ремонту.
- Управление движением самолетов: позволяет отслеживать и управлять движением самолетов на земле и в воздухе, а также оптимизировать маршруты и процессы посадки/взлета.
- Безопасность и аварийное реагирование: упрощает мониторинг безопасности на всей территории аэропорта, включая выявление потенциальных угроз и быстрое реагирование при чрезвычайных ситуациях.
- Анализ пассажиропотока: помогает анализировать движения пассажиров по терминалам, идентифицировать узкие места и оптимизировать размещение услуг.
- Экологический мониторинг: Слежение за влиянием аэропорта на окружающую среду, анализ выбросов и уровня шума.

Преимущества использования ГИС в управлении аэропортом:

- Увеличение эффективности: повышает точность планирования и управления благодаря детализированным пространственным данным и возможностям анализа.
- Снижение затрат: Оптимизация операций и внедрение инновационных решений помогает снизить эксплуатационные расходы.
- Поддержка принятия решений: обеспечивает руководство актуальной информацией, используемой для стратегического и оперативного планирования.

- **Приоритет безопасности:** способствует улучшению управления безопасностью как в аэропорту, так и в воздушном пространстве вокруг него.
- **Улучшение качества обслуживания:** Собранные данные позволяют повысить уровень услуг для пассажиров, сделав их путешествия более комфортными и безопасными.

Геоинформационная система управления аэропортом — это мощный инструмент, который помогает эффективно управлять функциями и процессами аэропорта. Она поддерживает безопасные и эффективные операции, отвечает на вызовы современного авиационного рынка и способствует улучшению качества обслуживания в условиях растущего пассажиропотока. С развитием технологий и доступностью данных ГИС будет играть все более значимую роль в будущем авиационной отрасли.

Подводя итоги анализа предметной области, мы изучили саму проблематику геоинформационных систем, и исходя из этой информации можем поставить задачи для проектирования самой системы:

- Система должна иметь возможность управления ресурсами аэропорта в рамках компетенций дирекции аэродромной:
 - Создание задач на обслуживание различных зон аэропорта.
 - Отслеживание перемещение техники, такой как самолеты и наземный обслуживающий транспорт.
 - Отображение графиков технических обслуживаний рейсов.
 - Отображение различного рода документаций для объектов аэропорта.
 - Отображение текущих задач на технике.
 - Отображение информации о рейсе при выборе ВС на карте.
 - Визуализация периодов занятости мест стоянок в виде диаграммы Ганта.
 - Отображение задач в виде диаграммы Ганта.

- Система должна иметь возможность персонализаций разного рода настроек для удобства диспетчера дирекции:
 - Индикация фильтров мест стоянок (например, занята ли он в текущий момент воздушным средством, или свободна).
 - Настройка системы уведомлений о создании задач (фильтрация по видам задач).
 - Индикация состояний задач – просрочка принятия, просрочка завершения и др.

2. Проектирование приложения

2.1. Выбор и обоснование средств разработки

2.1.1. Общая архитектура приложения

На данный момент в мире есть огромное количество подходов и архитектур к реализации приложений, но среди них можно выделить несколько основных:

Монолитная архитектура — это подход, при котором всё приложение разрабатывается и развёртывается как единое целое. Преимуществами такой архитектуры являются простота разработки и развёртывания, а также возможность быстрого масштабирования. Однако у монолитной архитектуры есть и недостатки, такие как сложность поддержки и обновления, а также невозможность горизонтального масштабирования.

Микросервисная архитектура — это подход, при котором приложение разбивается на небольшие, автономные сервисы. Каждый сервис отвечает за свою функциональность и может быть разработан, развёрнут и масштабирован независимо от других сервисов. Микросервисная архитектура позволяет достичь высокой гибкости и масштабируемости, но требует более сложной разработки и управления.

Сервис-ориентированная архитектура — это подход, при котором приложение разделяется на набор сервисов, которые могут взаимодействовать друг с другом через стандартизированные протоколы. SOA позволяет достичь высокой гибкости и масштабируемости, но также требует сложной разработки и управления.

Бессерверная архитектура — это подход, при котором приложение развёртывается на платформе, которая автоматически управляет инфраструктурой и масштабированием. Бессерверная архитектура позволяет сосредоточиться на разработке логики приложения, но может быть менее гибкой и масштабируемой, чем другие архитектуры.

Гибридная архитектура— это комбинация двух или более архитектур. Гибридная архитектура может сочетать преимущества разных подходов, но требует сложной разработки и управления.

Выбор архитектуры зависит от требований к приложению, его функциональности, масштабируемости, гибкости и других факторов. Каждая архитектура имеет свои преимущества и недостатки, поэтому важно выбрать ту, которая лучше всего подходит для конкретного проекта. Для моего проекта была выбрана микросервисная архитектура по следующим причинам:

- Возможность осуществления более гибкой разработки за счет разнесения функционала по «физически» независимым модулям;
- Высокий уровень гибкости при проектировке;
- Легкая поддержка – несмотря на то, что в целом принято считать, что микросервисы сложны в поддержке, при правильной дискретности, поддержка становится проще, за счет четкого разбиения ПО на независимые модули;
- Легкая читабельность – так же облегчение поддержки. Микросервисы по своей сути, представляются как набор небольших, но функциональных модулей. Меньше кодовой надстройки – проще понимать общую логику работы конкретного сервиса;
- Возможность удобного масштабирования – за счет своих небольших размеров, микросервисы проще организовывать в кластеры;

2.1.2 Выбор языка и стека разработки

Начать нужно с определения требований к стеку приложения. Их количество меняется в зависимости от типа и вида программного обеспечения, но к геоинформационной системе управления были указаны следующие требования:

1. Производительность и масштабируемость: - стек должен обеспечивать высокую производительность и масштабируемость приложения, чтобы оно могло обрабатывать большое количество запросов и пользователей. Стек должен быть способен поддерживать горизонтальное масштабирование, то есть распределение нагрузки между несколькими серверами.
2. Безопасность: - стек должен включать механизмы безопасности, такие как аутентификация, авторизация и шифрование данных, чтобы защитить приложение и пользовательскую информацию.
3. Надежность: - стек должен обеспечивать стабильную работу приложения и быстрое восстановление после сбоев. Должны быть предусмотрены механизмы резервного копирования и восстановления данных.
4. Совместимость: - стек должен поддерживать различные версии операционных систем, баз данных и других компонентов, чтобы обеспечить совместимость с различными окружениями.
5. Удобство разработки и поддержки: - стек должен предоставлять удобные инструменты для разработки, тестирования и поддержки приложения. Должны быть доступны готовые библиотеки и фреймворки, которые ускорят разработку и снизят вероятность ошибок.
6. Доступность: - стек должен обеспечивать доступность приложения, то есть его способность работать без сбоев и простоев.
7. Совместимость с другими системами: - приложение будет взаимодействовать с другими системами, необходимо убедиться в совместимости выбранного стека с этими системами.

8. Гибкость: - стек должен быть гибким и позволять легко вносить изменения в приложение, добавлять новые функции и улучшать производительность.

Исходя из этих требований, для разработки системы был выбран язык программирования Python третьей версии (конкретно 3.11). Ниже перечислены его преимущества перед другими языками программирования:

1. Читаемость кода - синтаксис языка разработан так, чтобы быть понятным и легко читаемым. Это позволяет программистам быстро понимать и поддерживать код, написанный другими разработчиками.
2. Большое сообщество – язык имеет большое и активное сообщество разработчиков, которые создают и поддерживают библиотеки и фреймворки для различных задач. Это облегчает поиск решений для возникающих проблем и ускоряет разработку.
3. Разнообразие фреймворков и библиотек – язык предлагает широкий выбор фреймворков и библиотек для разработки веб-приложений, научных вычислений, машинного обучения, анализа данных и других областей. Это позволяет разработчикам выбирать наиболее подходящие инструменты для своих задач.
4. Поддержка объектно-ориентированного программирования – язык поддерживает объектно-ориентированное программирование, что позволяет создавать модульный и расширяемый код.
5. Автоматизация и сценарии – Python 3 часто используется для написания сценариев и автоматизации задач. Это делает его полезным для создания простых утилит, обработки данных и других повседневных задач.
6. Масштабируемость – Python 3 может быть использован для создания как небольших проектов, так и крупных систем. Он позволяет разрабатывать масштабируемые приложения, способные обрабатывать большие объёмы данных и пользователей.

7. Поддержка функционального программирования – Python 3 также поддерживает функциональное программирование, что может быть полезно для некоторых задач.
8. Большое количество учебных материалов – существует множество учебных материалов, книг и онлайн-курсов, посвящённых Python 3. Это облегчает изучение языка для начинающих разработчиков.
9. Поддержка различных операционных систем. – язык доступен для различных операционных систем, включая Windows, macOS и Linux. Это позволяет разработчикам создавать приложения, работающие на разных платформах.
10. Поддержка множества парадигм программирования – язык поддерживает множество парадигм программирования, включая процедурное, объектно-ориентированное и функциональное программирование. Это позволяет разработчикам выбирать наиболее подходящий подход для своих задач.

Из главных недостатков можно выделить:

1. Производительность - из-за интерпретируемости Python может быть медленнее некоторых других языков программирования, таких как C++ или Java, особенно при работе с ресурсоёмкими задачами в 1 конкретном экземпляре приложения.
2. Проблемы с многопоточностью. Хотя Python поддерживает многопоточность, она может быть не так эффективна, как в других языках программирования, что может повлиять на производительность в некоторых случаях.

Однако на мой взгляд правильное разнесение функций микросервисов позволит устранить данные недостатки.

Основной фреймворк для разработки backend части приложения был выбран FastAPI. Это веб-фреймворк на языке Python, который упрощает разработку быстрых, эффективных и масштабируемых веб-приложений. Он

предоставляет набор инструментов и функций для создания API, включая автоматическую генерацию документации, встроенную поддержку JSON-схем и другие полезные функции.

Основные преимущества FastAPI:

1. Высокая производительность: FastAPI использует асинхронную обработку запросов, что позволяет обрабатывать большое количество запросов одновременно и улучшает общую производительность приложения.
2. Простота разработки: FastAPI предоставляет интуитивно понятный и простой синтаксис, который упрощает написание кода и уменьшает количество шаблонного кода.
3. Автоматическая документация: FastAPI автоматически генерирует документацию для вашего API на основе аннотаций Python, что экономит время и усилия на написание документации вручную.
4. Встроенная поддержка JSON-схем: FastAPI интегрируется с библиотекой, которая позволяет легко определять и проверять схемы JSON, что улучшает безопасность и надежность данных.
5. Встроенная поддержка асинхронных баз данных: FastAPI поддерживает работу с асинхронными базами данных, такими как PostgreSQL и SQLite, что позволяет создавать высокопроизводительные и масштабируемые приложения.
6. Интеграция с другими библиотеками и фреймворками: FastAPI легко интегрируется с другими популярными библиотеками и фреймворками Python, такими как SQLAlchemy, Flask-Migrate и Django Rest Framework, что позволяет использовать их в вашем проекте по мере необходимости.
7. Поддержка HTTP/2: FastAPI поддерживает HTTP/2, что позволяет улучшить производительность и безопасность ваших приложений.
8. Поддержка Swagger: FastAPI интегрируется со спецификацией OpenAPI (Swagger), что позволяет легко документировать и тестировать ваш API.

9. Удобное тестирование: FastAPI предоставляет встроенные инструменты для тестирования API, такие как, что упрощает написание и запуск тестов.

Так же стоит упомянуть про другой популярный фреймворк – Django. Это фреймворк для создания веб-приложений на языке Python. Он предоставляет множество функций и инструментов для разработки, но у него есть и некоторые недостатки, которые могут сделать FastAPI более предпочтительным выбором в некоторых случаях. Конкретно в случае данной работы – FastAPI является более легковесным фреймворком, за счет чего больше подходит для организации микросервисной архитектуры. Django же является монолитным фреймворком, он безусловно удобен, когда необходимо создать прототип, но в случае production-разработки на микросервисах безусловно лидерство за FastAPI по следующим причинам:

1. Производительность. FastAPI использует асинхронную обработку запросов, что позволяет быстрее обрабатывать большое количество запросов и улучшает общую производительность приложения.
2. Простота разработки. Синтаксис FastAPI более интуитивный и простой, что упрощает написание кода и сокращает количество шаблонного кода.
3. Автоматическая документация. FastAPI автоматически генерирует документацию для API на основе аннотаций Python, что экономит время и усилия на написание документации.
4. Встроенная поддержка JSON-схем. FastAPI интегрируется с библиотекой Marshmallow, которая позволяет легко определять и проверять схемы JSON, улучшая безопасность и надёжность данных.

Для хранения данных были выбраны следующие базы данных MongoDB, PostgreSQL и как кэш-база Redis. Это довольно мощные решения для больших и нагруженных проектов, ниже мы рассмотрим их основные характеристики, преимущества и недостатки.

PostgreSQL (часто сокращённо Postgres) — это свободная объектно-реляционная система управления базами данных (СУБД), которая поддерживает стандарт SQL. Эта СУБД отличается высокой надёжностью, масштабируемостью и безопасностью данных.

Основные характеристики PostgreSQL:

1. Открытый исходный код. Система управления базами данных доступна для свободного использования, модификации и распространения. Это позволяет разработчикам адаптировать PostgreSQL под свои нужды и исправлять возможные ошибки.
2. Поддержка стандарта SQL. PostgreSQL поддерживает множество стандартов SQL, что обеспечивает совместимость с другими системами управления базами данных.
3. Объектно-реляционная модель. PostgreSQL использует объектно-реляционную модель данных, что позволяет хранить различные типы данных в одной базе данных.
4. Масштабируемость. Система управления базами данных способна обрабатывать большие объёмы данных и поддерживать высокую нагрузку.
5. Надёжность. PostgreSQL обеспечивает сохранность данных и гарантирует их доступность.
6. Безопасность. PostgreSQL предоставляет различные механизмы для защиты данных, включая шифрование и управление доступом.
7. Гибкость. Система управления базами данных позволяет настраивать различные параметры для оптимизации производительности.
8. Поддержка сообщества. PostgreSQL имеет активное сообщество разработчиков, которые постоянно улучшают систему и создают новые функции.

PostgreSQL используется в различных областях, включая веб-разработку, научные исследования, финансовые операции и другие. Эта система

управления базами данных отличается высокой производительностью, надёжностью и безопасностью данных.

Основные преимущества PostgreSQL:

- Поддержка широкого спектра функций SQL. Это позволяет использовать все возможности языка запросов для работы с данными.
- Гибкость настроек. Пользователи могут настраивать параметры PostgreSQL в соответствии с требованиями своего проекта.
- Высокая надёжность. Система управления базами данных обеспечивает сохранность данных и гарантирует их доступность.
- Безопасность. PostgreSQL предоставляет различные механизмы для защиты данных, включая шифрование и управление доступом.
- Активное сообщество. Это позволяет быстро получать поддержку и решения возникающих проблем.

Исходя из этих фактов, мы выбрали PostgreSQL как одну из вариаций базы данных для нашего проекта. Однако, геоинформационная система подразумевает хранение геоинформации, а стандартная версия PostgreSQL не предоставляет возможность их хранить в удобном и индексируемом виде. Для решения этой проблемы было решено использовать расширение PostGIS.

PostGIS — это мощное расширение для системы управления базами данных PostgreSQL, предназначенное для работы с географическими объектами и пространственными данными. Оно позволяет хранить, обрабатывать и анализировать геоданные, такие как координаты точек, линий, полигонов и других геометрических объектов.

PostGIS поддерживает множество различных геометрических типов данных, включая:

- Point: точка на плоскости (широта и долгота).
- Line String: линейная строка (например, дорога или река).
- Polygon: многоугольник (например, границы стран или регионов).
- MultiPoint: множество точек.
- MultiLine String: множество линейных строк.

- MultiPolygon: множество многоугольников.
- Geometry Collection: коллекция геометрических объектов.

PostGIS предоставляет широкий набор операторов и функций для выполнения различных операций с пространственными данными, таких как:

- Intersection: пересечение двух геометрических объектов.
- Union: объединение двух геометрических объектов.
- Difference: разность двух геометрических объектов.
- Buffer: создание буферной зоны вокруг геометрического объекта.
- Distance: вычисление расстояния между двумя точками.
- Convex Hull: вычисление выпуклой оболочки для множества точек.

PostGIS легко интегрируется с другими расширениями PostgreSQL, такими как GIS Toolkit. GIS Toolkit предоставляет дополнительные инструменты и функции для работы с гео данными, что позволяет расширить функциональность PostGIS и решать более сложные задачи.

Таким образом, мы экономим время обработки геоинформации в базе данных. Перейдем к еще одному виду БД для нашего проекта – MongoDB:

MongoDB — это документо-ориентированная система управления базами данных (СУБД), которая хранит данные в формате JSON-подобных документов. Она отличается высокой масштабируемостью и гибкостью, что делает её подходящей для широкого спектра приложений.

Основные характеристики MongoDB:

1. Документо-ориентированная модель данных. MongoDB хранит данные в формате BSON (Binary JSON), который похож на JSON, но поддерживает двоичные данные. Это позволяет хранить данные в более естественном и интуитивно понятном формате, что упрощает разработку и поддержку приложений.
2. Масштабируемость. MongoDB способна обрабатывать большие объёмы данных и поддерживать высокую нагрузку. Это достигается за счёт горизонтальной масштабируемости, то есть возможности добавления новых серверов для распределения нагрузки.

3. Гибкость. MongoDB позволяет настраивать различные параметры для оптимизации производительности. Это включает в себя настройку индекса, размера буфера и других параметров.
4. Высокая доступность. MongoDB обеспечивает сохранность данных и гарантирует их доступность. Это достигается за счёт репликации данных на несколько серверов.
5. Безопасность. MongoDB предоставляет различные механизмы для защиты данных, включая шифрование и управление доступом.
6. Гибкость запросов. MongoDB поддерживает широкий спектр запросов, включая запросы по диапазону, регулярные выражения и другие. Это позволяет выполнять сложные запросы к данным.

Однако, как и любая другая система управления базами данных, MongoDB имеет свои недостатки и может не подходить для всех проектов. Выбор СУБД зависит от конкретных требований проекта и его особенностей.

Основные преимущества MongoDB:

- Документо-ориентированная модель данных. Это позволяет хранить данные в более естественном и интуитивно понятном формате.
- Горизонтальная масштабируемость. Это позволяет распределять нагрузку на несколько серверов.
- Гибкость запросов. Это позволяет выполнять сложные запросы к данным.
- Активное сообщество. Это обеспечивает поддержку и решения возникающих проблем.
- Поддержка широкого спектра приложений. Это делает MongoDB подходящей для различных проектов.

Однако MongoDB также имеет некоторые недостатки:

- Сложность настройки. Это может быть сложно для начинающих пользователей.
- Ограниченная поддержка транзакций. Это может быть недостатком для некоторых приложений.

- Некоторая несовместимость с другими системами. Это может создавать проблемы при интеграции с другими системами.

Может возникнуть вопрос – в чем необходимость использования 2 разных баз данных – затем что в некоторых сервисах требуется поддержка транзакций БД для обеспечения консистентности данных, и как раз для этой задачи и используется PostgreSQL.

Redis — это система управления базами данных (СУБД), которая хранит данные в памяти и на диске. Она поддерживает различные типы данных и предоставляет широкий спектр возможностей для работы с ними.

Основные характеристики Redis:

1. Хранение данных в памяти и на диске. Redis хранит данные в оперативной памяти (RAM), что обеспечивает быстрый доступ к ним. Однако для сохранения данных при перезапуске сервера или сбое питания Redis также использует диск.
2. Поддержка различных типов данных. Redis поддерживает различные типы данных, включая строки, хэши, списки, множества и отсортированные множества. Это позволяет хранить и обрабатывать данные различных форматов.
3. Быстрый доступ к данным. Благодаря хранению данных в памяти, Redis обеспечивает быстрый доступ к ним. Это делает Redis подходящим для приложений, требующих высокой производительности.
4. Поддержка кластеризации. Redis поддерживает кластеризацию, что позволяет распределять данные и нагрузку между несколькими серверами. Это обеспечивает высокую доступность и масштабируемость.
5. Поддержка репликации. Redis поддерживает репликацию, что обеспечивает сохранность данных и защиту от сбоев. Это позволяет создавать резервные копии данных и обеспечивать их доступность.

6. Поддержка транзакций. Redis поддерживает транзакции, что позволяет гарантировать целостность данных при выполнении нескольких операций.

Так же для удобного обмена данными между сервисами будет использоваться брокер сообщений Kafka. Рассмотрим, что такое брокер сообщений и какие они бывают.

Брокеры сообщений — это программное обеспечение, которое обеспечивает взаимодействие между различными системами и компонентами программного обеспечения через обмен сообщениями. Они играют важную роль в распределённых системах, где требуется координация и обмен данными между различными компонентами.

Основные функции брокеров сообщений:

1. Обмен сообщениями: брокеры сообщений обеспечивают передачу сообщений между производителями (отправителями) и потребителями (получателями) сообщений. Это позволяет компонентам программного обеспечения обмениваться данными без необходимости прямого взаимодействия.
2. Хранение сообщений: брокеры сообщений могут временно хранить сообщения, чтобы обеспечить надёжную доставку даже в случае сбоя в системе. Это позволяет гарантировать, что сообщения будут доставлены получателям, даже если отправитель или получатель временно недоступны.
3. Управление очередями: некоторые брокеры сообщений позволяют создавать и управлять очередями сообщений. Это может быть полезно для организации обмена данными между компонентами, которые должны обрабатывать сообщения в определённом порядке или с определёнными приоритетами.
4. Транзакции: некоторые брокеры сообщений поддерживают транзакции, что позволяет гарантировать доставку или не доставку сообщений в зависимости от успешного выполнения транзакции.

Популярные брокеры сообщений:

- Apache Kafka — один из самых популярных брокеров сообщений, который используется в различных проектах, таких как Twitter, LinkedIn и Spotify.
- RabbitMQ — ещё один популярный брокер сообщений, который используется в различных проектах, таких как Facebook, Uber и Netflix.

Мы выбрали брокер Apache Kafka по следующим причинам:

1. Высокая пропускная способность. Kafka может обрабатывать большие объёмы данных и масштабироваться горизонтально, что позволяет системе расти вместе с увеличением нагрузки.
2. Гарантированная доставка сообщений. Kafka обеспечивает доставку сообщений в порядке их отправки и гарантирует, что ни одно сообщение не будет потеряно.
3. Поддержка различных форматов данных. Kafka может работать с различными форматами данных, включая JSON, Avro, Protobuf и другие.
4. Горизонтальное масштабирование. Kafka может быть масштабирована горизонтально путём добавления новых брокеров, что позволяет распределять нагрузку между ними.
5. Поддержка нескольких потребителей. Kafka позволяет одновременно работать с данными нескольким потребителям, что повышает эффективность системы.
6. Поддержка резервного копирования. Kafka предоставляет возможность резервного копирования данных, что обеспечивает защиту от потери информации.
7. Поддержка управления доступом. Kafka позволяет управлять доступом к данным, что обеспечивает безопасность системы.
8. Интеграция с другими системами. Kafka может интегрироваться с другими системами обмена сообщениями, что позволяет создавать сложные распределённые системы.

После определения основного стека для backend части приложения, который полностью отвечает изначальным требованиям мы переходим к описанию интерфейса приложения и обоснованию его технологического стека.

2.2 Описание интерфейса приложения

На сегодняшний день имеется огромное количество разных подходов и практик к построению интерфейса приложения, но также как и с backend частью, к интерфейсу предъявляются следующие требования:

1. Интуитивно понятный и простой дизайн: интерфейс должен быть понятным и удобным для пользователя, чтобы он мог легко найти нужную информацию и функции.
2. Единый стиль оформления: все элементы интерфейса должны соответствовать общему стилю приложения. Это включает в себя цветовую палитру, шрифты, иконки и анимацию.
3. Адаптивность: интерфейс должен корректно отображаться на различных устройствах и разрешениях экранов.
4. Отзывчивость: интерфейс должен быстро реагировать на действия пользователя, чтобы обеспечить комфортное взаимодействие.
5. Цветовая гамма: цвета интерфейса должны быть приятными для восприятия и соответствовать общему стилю приложения. Рекомендуется использовать не более 3-4 цветов.
6. Иконки и изображения: иконки и изображения должны быть качественными и соответствовать контексту приложения.
7. Текстуры и эффекты: использование текстур и эффектов может улучшить восприятие интерфейса, но не должно быть избыточным.
8. Навигация: интерфейс должен иметь понятную и удобную навигацию, чтобы пользователь мог легко перемещаться между разделами приложения.
9. Отзывчивость на действия пользователя: интерфейс должен мгновенно реагировать на действия пользователя, чтобы обеспечить комфортное взаимодействие.
10. Минимализм: интерфейс должен быть минималистичным и не перегруженным элементами. Это поможет избежать путаницы и упростит взаимодействие с приложением.

- 11.Согласованность: все элементы интерфейса должны быть согласованы между собой. Это включает в себя стиль оформления, цвета, шрифты и т. д.
- 12.Доступность: интерфейс должен быть доступен для пользователей с ограниченными возможностями. Это включает в себя поддержку различных устройств, языков и возможностей пользователей.
- 13.Удобство использования: интерфейс должен быть удобным и комфортным для пользователя. Это включает в себя простоту навигации, доступность функций и удобство взаимодействия с элементами интерфейса.

Исходя из этих требований, был сделан выбор в пользу веб версии приложения. На сегодняшний день веб приложения имеют ряд серьезных преимуществ перед десктопными:

1. Доступность. Веб-приложения доступны с любого устройства, подключённого к интернету, будь то компьютер, планшет или смартфон. Это позволяет пользователям получать доступ к своим данным и функциональности приложения в любое время и в любом месте, где есть интернет.
2. Отсутствие необходимости установки. Веб-приложения не требуют установки на устройство пользователя, что экономит время и упрощает доступ к приложению.
3. Обновления без участия пользователя. Разработчики могут выпускать обновления веб-приложения без участия пользователя. Это гарантирует, что приложение всегда будет работать корректно и безопасно.
4. Централизованное хранение данных. Веб-приложения хранят данные на серверах, что обеспечивает их сохранность и доступность в случае поломки устройства или его утери.
5. Масштабируемость. Веб-приложения могут легко масштабироваться для обработки большого количества пользователей и данных. Это

позволяет им работать быстро и эффективно, даже при высоких нагрузках.

6. Совместимость. Веб-приложения работают на разных операционных системах и устройствах, что обеспечивает их доступность для широкого круга пользователей.
7. Экономическая эффективность. Разработка и поддержка веб-приложений обычно дешевле, чем разработка и поддержка десктопных приложений, особенно для компаний с большим количеством пользователей.
8. Безопасность. Веб-приложения могут использовать современные методы шифрования и аутентификации для обеспечения безопасности данных пользователей.
9. Кроссплатформенность. Веб-приложения могут быть доступны на разных платформах без необходимости создания отдельных версий для каждой из них. Это упрощает разработку и поддержку приложения.
10. Возможность использования облачных сервисов. Веб-приложения могут использовать облачные сервисы для хранения данных и обработки запросов пользователей. Это обеспечивает высокую доступность и масштабируемость приложения.

Для реализации приложения был выбран фреймворк React. Это JavaScript-библиотека с открытым исходным кодом, которая используется для создания интерактивных пользовательских интерфейсов. Она позволяет разработчикам создавать компоненты, которые можно повторно использовать в разных частях приложения.

Основные преимущества React:

1. Виртуальный DOM (Document Object Model). React использует виртуальный DOM, который представляет собой копию реального DOM (Document Object Model). Это позволяет быстрее обновлять интерфейс, так как React сравнивает виртуальный DOM с реальным и применяет только необходимые изменения.

2. JSX (JavaScript XML). JSX — это расширение JavaScript, которое позволяет писать HTML-подобный код в JavaScript. Это упрощает создание компонентов и улучшает читаемость кода.
3. Компонентный подход. React использует компонентный подход, который позволяет создавать независимые и переиспользуемые компоненты. Это упрощает разработку и поддержку кода.
4. Однонаправленный поток данных. React использует однонаправленный поток данных, который упрощает отслеживание изменений и предотвращает путаницу в коде.
5. Сообщество и экосистема. React имеет большое и активное сообщество разработчиков, которые создают множество инструментов и библиотек для работы с React.
6. Производительность. React позволяет создавать быстрые и отзывчивые пользовательские интерфейсы, что важно для любого приложения.
7. Гибкость. React можно использовать как для небольших проектов, так и для крупных приложений. Он подходит для разработки мобильных приложений, веб-сайтов и других проектов.

После определения стека для backend и frontend приложений, мы можем приступить к непосредственной реализации.

3. Разработка приложения

3.1. Процесс разработки приложения

3.1.1 Backend часть приложения

Как было указано во второй главе, для нашего приложения мы решили использовать микросервисную архитектуру.

На Рисунке 1 представлена схема сервисов ГИСа:

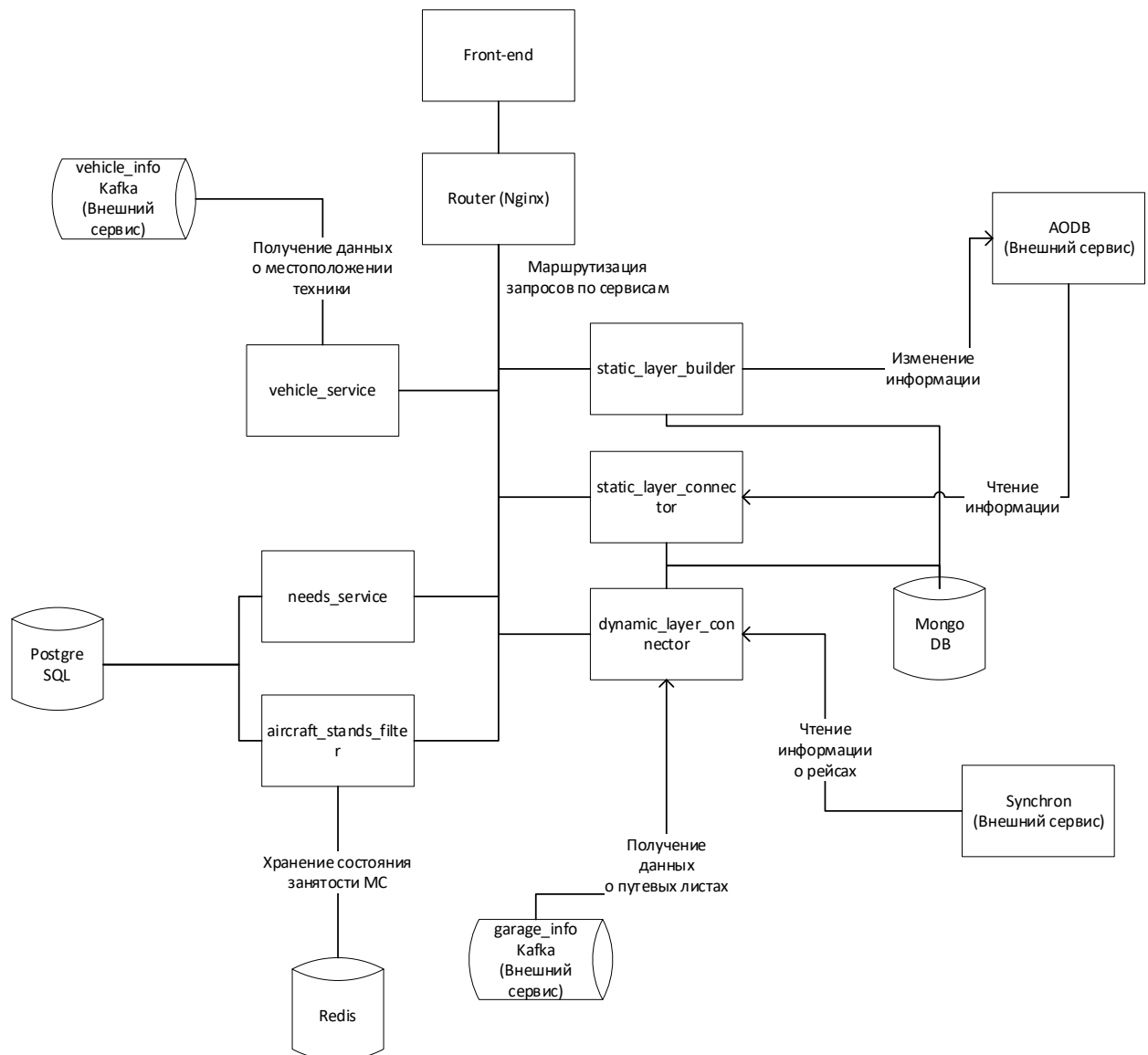


Рисунок 1 - Общая архитектура ГИС

Начать нужно с того, что front-end общается с микросервисами, разделенными по функционалу. При обращении к Router, его запросы маршрутизируются на соответствующий сервис, и после обработки

возвращаются обратно. Далее идет перечисление сервисов, и рассказывается об их назначении и принципах работы.

Начнем с сервисов, которые являются сторонними (внешними, значит что были сделаны другими командами, либо не являются нашей разработкой вовсе)

Сервис Router – просто контейнер с web-сервисом Nginx на борту. Уже готовое и популярное решение для обработки входящих запросов. Nginx — это свободный и открытый веб-сервер и обратный прокси-сервер, а также сервер для быстрой отдачи статических файлов и почтовый сервер. Он написан на С и использует неблокирующий метод обработки соединений, что позволяет обрабатывать большое количество одновременных подключений.

Nginx широко используется в веб-разработке благодаря своей высокой производительности, эффективности и низкому потреблению ресурсов. Он может работать как самостоятельный сервер или как обратный прокси-сервер перед другими серверами (например, веб-серверами, серверами приложений и базами данных).

Сервис AODB – Airport Operational Database, база данных в которой хранится вся справочная информация системы ГИС.

Сервис Synchron – «легаси» проект аэропорта Шереметьево. Хранит информацию об рейсах, данных мест стоянок и др.

Теперь переходим к сервисам, которые были непосредственно реализованы нашей командой для ГИС:

Сервис Static Layer Connector – используется для работы с данными из AODB. Был создан для уменьшения кол-ва запросов непосредственно в хранилище. Его основная задача – отдавать данные «слоев». Это сущность, которая объединяет в себе элементы по определенному бизнес признаку.

Слой характеризуется следующими параметрами:

- Общие параметры - Параметры, которые характеризуют ключевые характеристики слоя:

- Параметры задач - параметры, предназначенные для создания шаблона потребности (комментарий, файлы) - сущность SLC;
- Иерархия - параметр для отображения списка объектов;
- Условное форматирование - параметр для визуального отображения объекта;
- Фильтры - параметры для отображения списка объектов;
- Элементы слоя (для сервисов SLC и SLB)
- Элемент слоя - сущность, которая содержит в себе информацию о гео-объекте системы

Сервис Static Layer Builder – данный сервис позволяет производить непосредственную работу со слоями, их редактирование, добавление и удаление. На Рисунке 2 представлена диаграмма последовательности работы логики сервиса в общем виде:

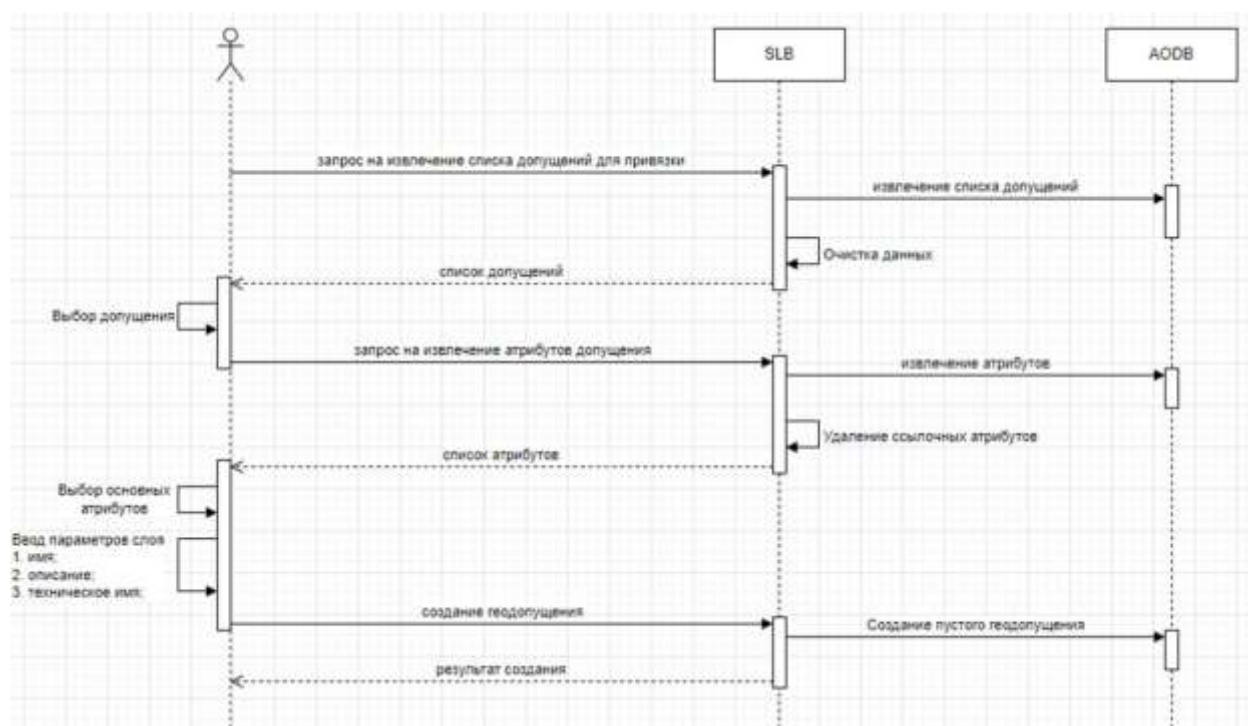


Рисунок 2 - Общая логика работы сервиса SLB

На Рисунке 3 и 4 представлены диаграммы последовательности работы сервиса:

Создания гео-слоя

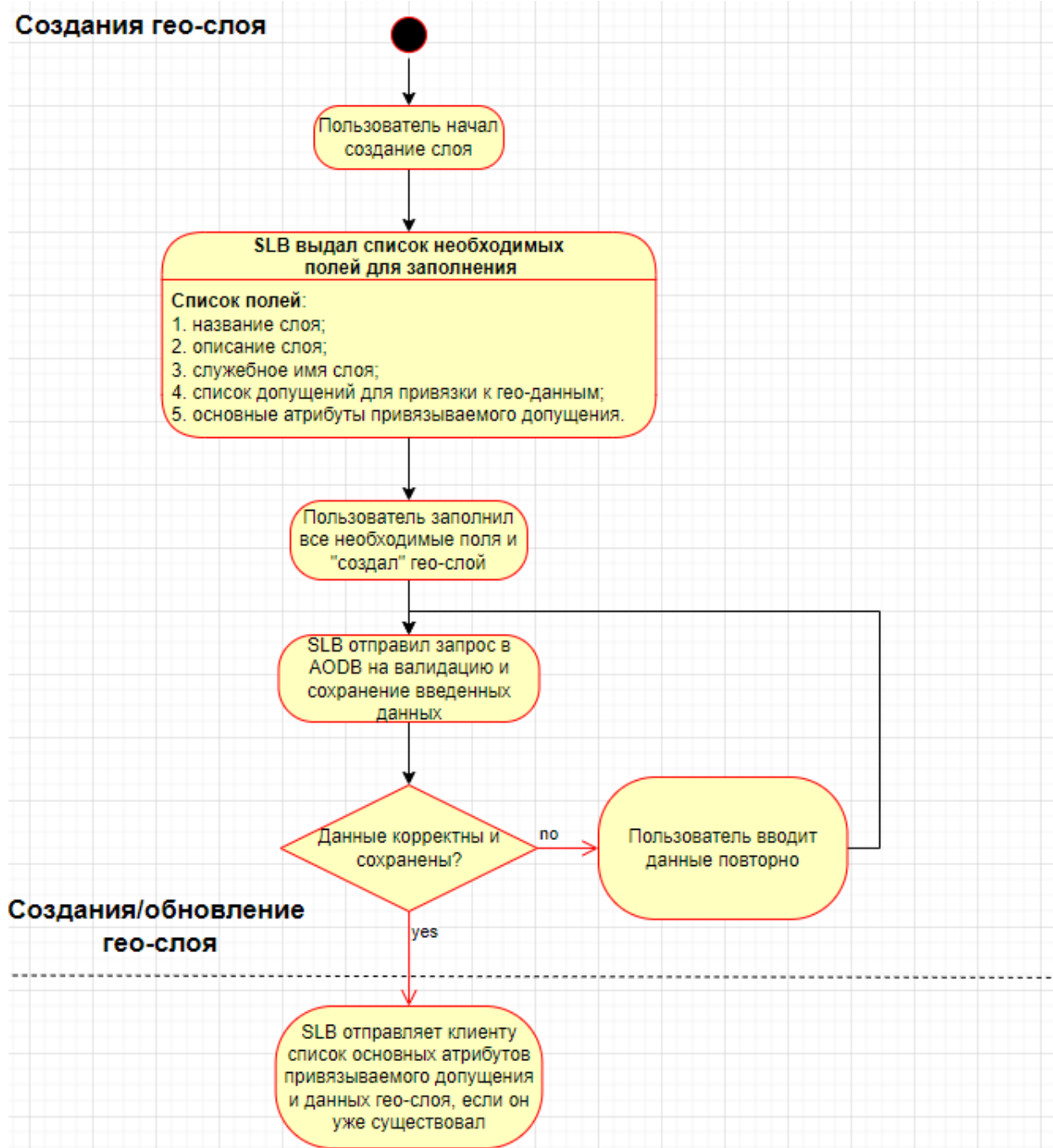


Рисунок 3 - Создание гео-слоя

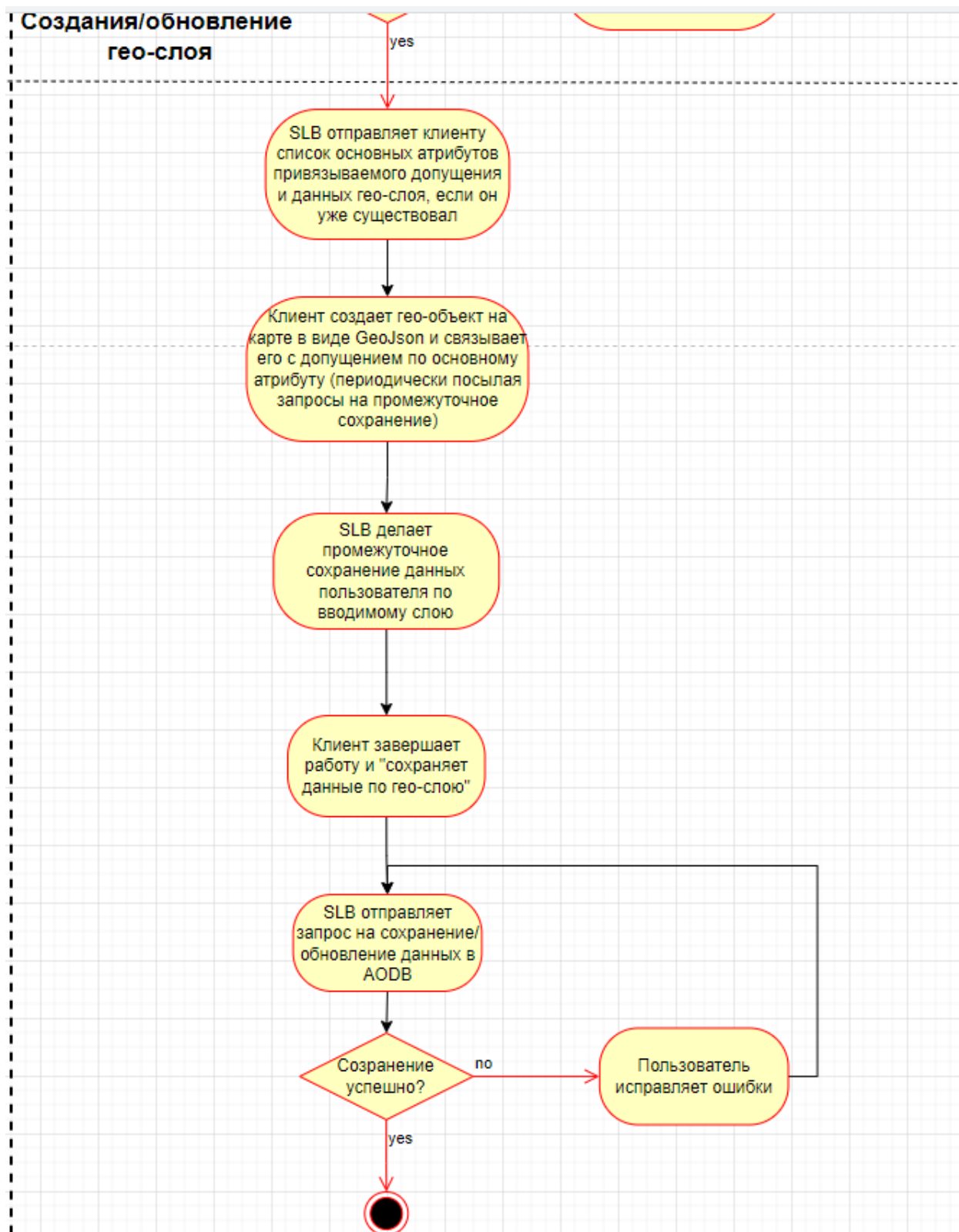


Рисунок 4 - Создание и обновление гео-слоя

Сервис Dynamic Layer Service – отвечает за обработку «динамических слоев». Хранит в себе информацию о ТС, а также получает информацию о назначении ТС на смену работы.

Сервис Aircraft Stands Filters – отвечает за условное форматирование МС. Это помогает диспетчерам определять статус МС. Ниже идет пояснение, что такое фильтр, и как он работает.

Фильтр — это сущность, которая объединяет в себе правила условного форматирования для конкретного слоя;

Системный фильтр (system) — это фильтр, который создается в автоматическом режиме системой ГИС и служит фильтром по умолчанию, если нет иных созданных фильтров. Может быть только в единственном экземпляре для каждого слоя;

Общий фильтр (shared) — это фильтр, который создается администратором своего подразделения и является общедоступным для всех, кто входит в данное подразделение;

Личный фильтр (personal) — это фильтр, который создается пользователем системы и имеет индивидуальный формат взаимодействия.

Фильтры по природе создания делятся на три типа:

1. Системный:

- Используется в фильтрах МС;
- Создается администратором системы ГИС;
- Глобальный для слоя и не зависит от ролей;
- Является активным по умолчанию, если не выбрано иного;

2. Общий и личный:

- Данные фильтры всегда имеют привязку к дирекции в которую входит пользователь;
- Активность конкретного фильтра определяется конкретным пользователем;

3. Ролевые характеристики фильтров:

- Системный - R - всем; CUD - администратор gis системы;
- Общий- R - сотрудники дирекции; CUD - администратор дирекции;

- Личный - CRUD - конкретный сотрудник дирекции;

На Рисунке 5 представлена диаграмма взаимодействия с фильтрами:

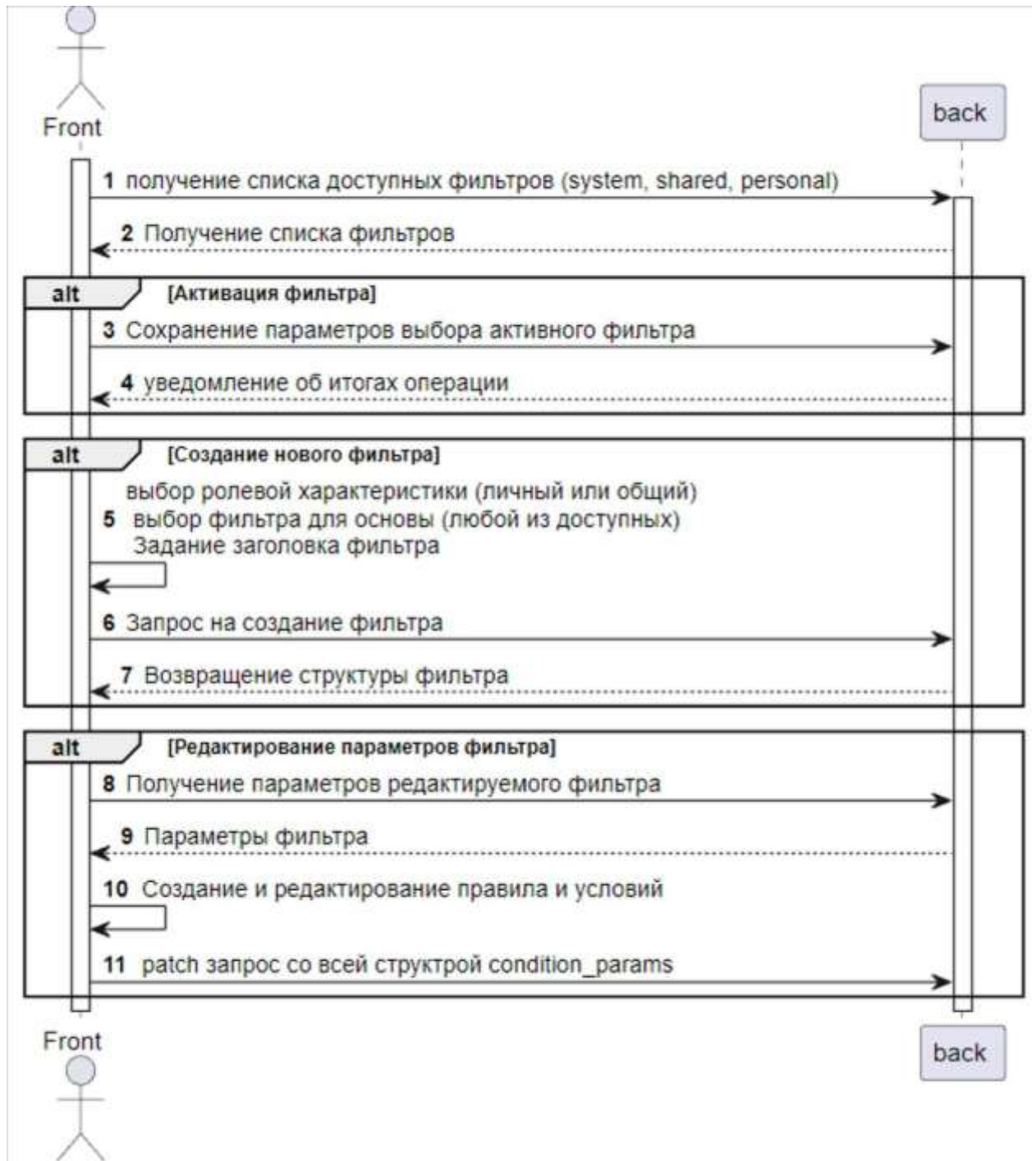


Рисунок 5 - Диаграмма взаимодействия

Далее, созданный фильтр должен пройти через модуль условного форматирования для того, чтобы мы поняли, какое правило фильтра сработало. Сам фильтр состоит из условий, которые образуют правила.

Сущности условий:

1. Правила - основные компоненты, которые определяют причастность к тому или иному формату. Правила состоят из условий и формата;
2. Условия - являются единицей конкретного правила;
3. Оператор правила - соединительный элемент между условиями одного правила;
4. База условия - задающий атрибут условия, определяющий тип операций и его значения;
5. Значение условия - задаваемое значение тип которого определяется базой условия;
6. Оператор условия - один из операторов сравнения, который участвует в одном конкретном условии;
7. Атрибуты условия - набор сущностей, предопределенных конкретным сервисом и слоем, которые могут быть использованы в качестве Базы условия
8. Элемент правила - одна из сущностей условия или оператора правила
9. Формат правила - настройки форматирования при соблюдении условий правила

Разновидности сущностей:

1. Оператор правила: (,) , И , ИЛИ;
2. База условия определяет значения условия и разделяется на следующие типы значений:
 - Время;
 - Число;
 - Строка;
 - Булево;
3. Значение условия определяется типом базы условия и задается из списка предлагаемых значений или ручным вводом;
4. Оператор условия: =, !=, >, <, <=, >=;

Сервис Needs service – сервис потребностей и поручений, который занимается обработкой задач. Потребность – это задача, включающая в себя N кол-во работ, которые должны быть произведены. Поручения в свою очередь – это набор потребностей. Такое разделение было создано для выполнения разных требований к самой системе, например одна дирекция пользуется только потребностями, а другая дирекция может создавать задачи первой, и эти задачи и называются поручениями. В главе 3.1.2 приводятся примеры этих задач, и то, как они выглядят.

Сервис Vehicle service – сервис, отвечающий за взаимодействие с инфраструктурой аэропорта по части отслеживания местоположения воздушных судов и наземной техники, позволяет диспетчеру в реальном времени видеть оперативную ситуацию.

3.1.2 Frontend часть приложения

Общий вид интерфейса геоинформационной системы аэропорта представлен на Рисунках 6, 7:

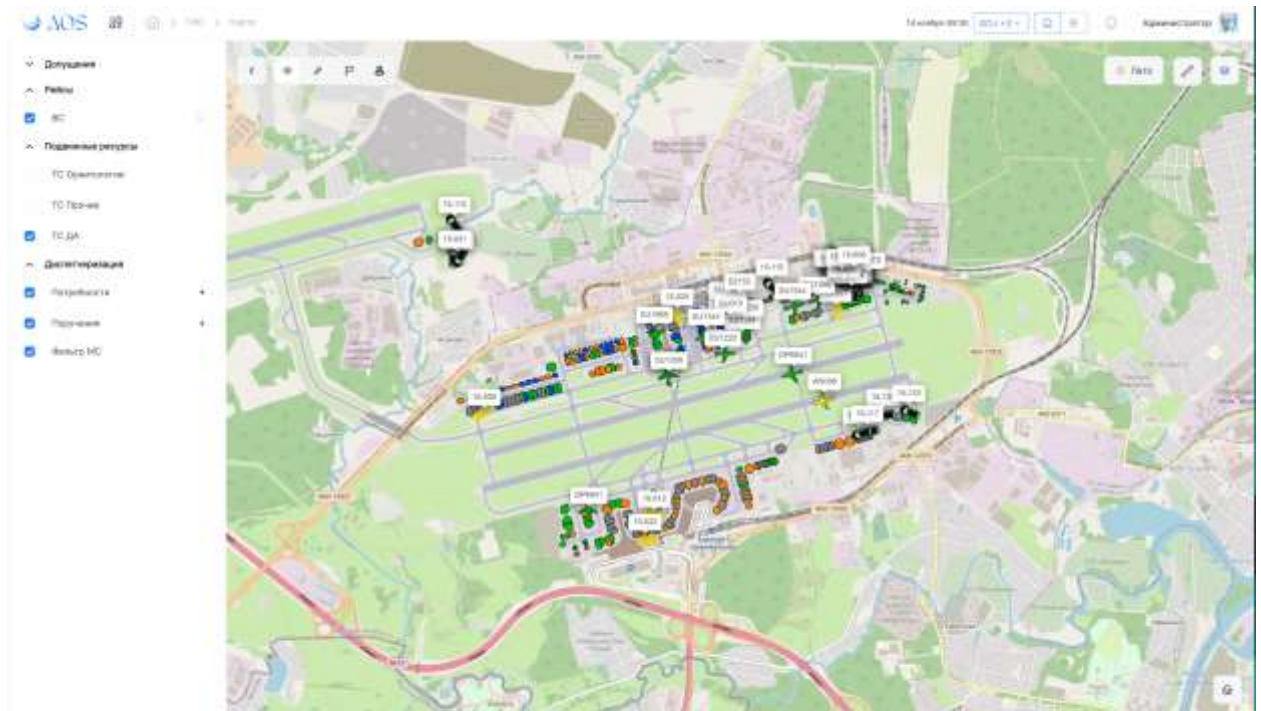


Рисунок 6 - Общий вид интерфейса



Рисунок 7 - Общий вид интерфейса

На данных рисунках в левом баре отображается список слоев. Каждый из этих слоев является определённой группировкой объектов аэропорта, например,

МС – это Места Стоянок для самолетов, РД – рулевая дорожка и т.д. При нажатии правой кнопки мышки по месту стоянки, открывается еще 2 бара: снизу диаграмма Ганта по оперативной занятости места стоянки, на нем видно отображение прилетных и вылетных рейсов, а также общая продолжительность занятия места стоянки. Справа же открывается информация по текущему занятию: прилетной и вылетной рейс, время занятия места стоянки, отклонения от запланированного времени. Так же можно посмотреть технический график обслуживания самолета, если нажать на иконку в виде гаечного ключа. Пример отображен на Рисунке 8:

Статус выполнения ТГО [REDACTED]

 SU427
 SU422

Наименование операции	Начало			Окончание			Продолжительность		
	План	Факт	Откл	План	Факт	Откл	План	Факт	Откл
Прибытие ВС				22:13	22:11	-00:02			
Подгон трапа				22:15	22:15	00:00			
Подкл. источника эл. питания				22:15	22:16	+00:01			
Установка колодок				22:16	22:13	-00:03			
Высадка пассажиров	22:14	22:15	+00:01	22:26	22:26	00:00	00:12	00:11	00:01
Разгрузка почты, груза	22:17		+02:51	22:45		+02:23	00:28		
Разгрузка багажа		22:17		22:45	22:40	-00:05		00:23	
Уборка ВС				22:48		+02:20			
Уборка, экипировка ВС				22:48		+02:20			
Отгон трапа				22:50	22:51	+00:01			
Откл. источника эл. питания				22:53	22:51	-00:02			
Слив воды		22:30			23:06			00:37	
Прибытие баг. грузчиков									
Отъезд 1 партии багажа									
Загрузка б/питания									
Выключение двигателей									
Подгон второго трапа									
Открытие груз. люков									
Посадка пассажиров									

Рисунок 8 - Таблица ТГО

На нем видены времена выполнения каждой из задач, а также видны временные отклонения.

Если нажать на самолет правой кнопкой мыши, то высветится информация по рейсу, представленная на Рисунке 9:

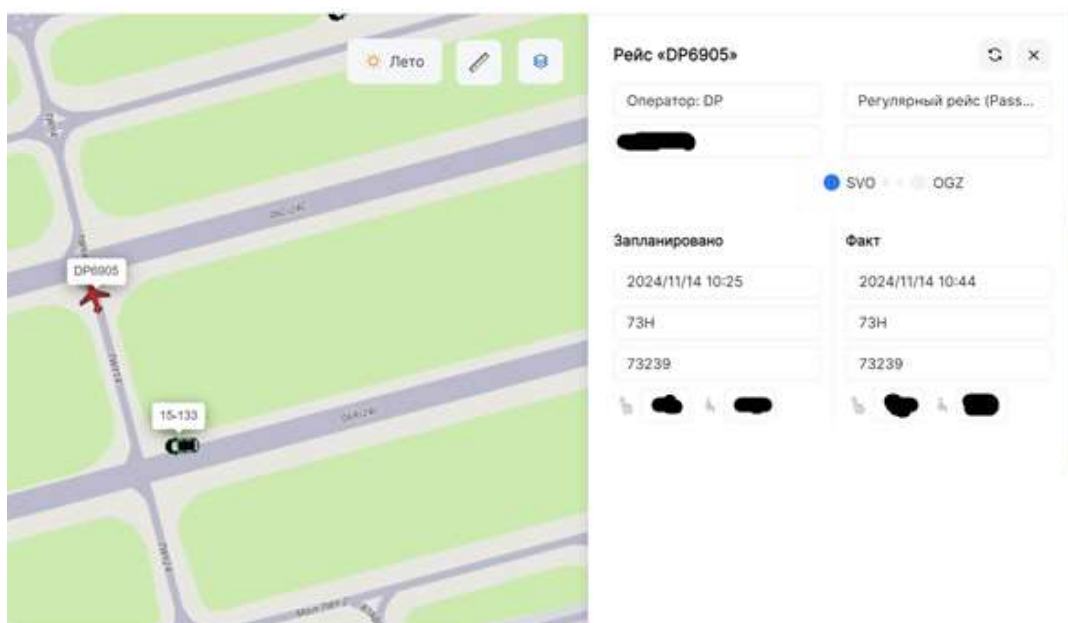


Рисунок 9 - Информация по рейсу

При нажатии правой кнопкой мыши на машинку, высветится информация о технике, и человеке, который в данный момент на ней находится. Это представлено на Рисунке 10:

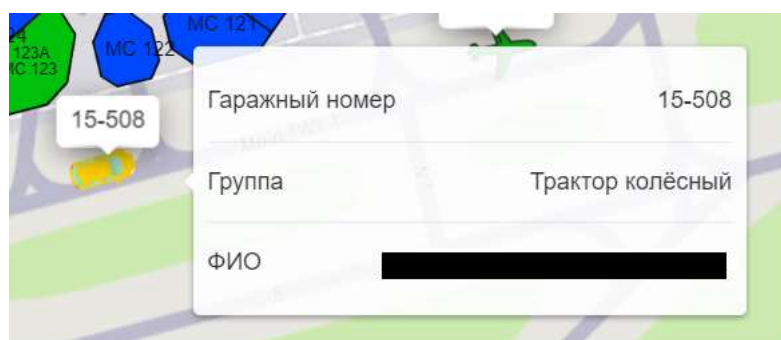


Рисунок 10 - Информация о наземной технике

Создание потребности – процесс назначения задачи для водителей наземной техникой, используется Дирекцией Аэродромной. Особенно актуально зимой, так как места стоянки и прочая инфраструктура нуждаются в регулярной очистке от снега. Для создания потребности выбирается флажок

в верхнем меню на карте, и выбирается объект задачи, как это представлено на Рисунке 11:

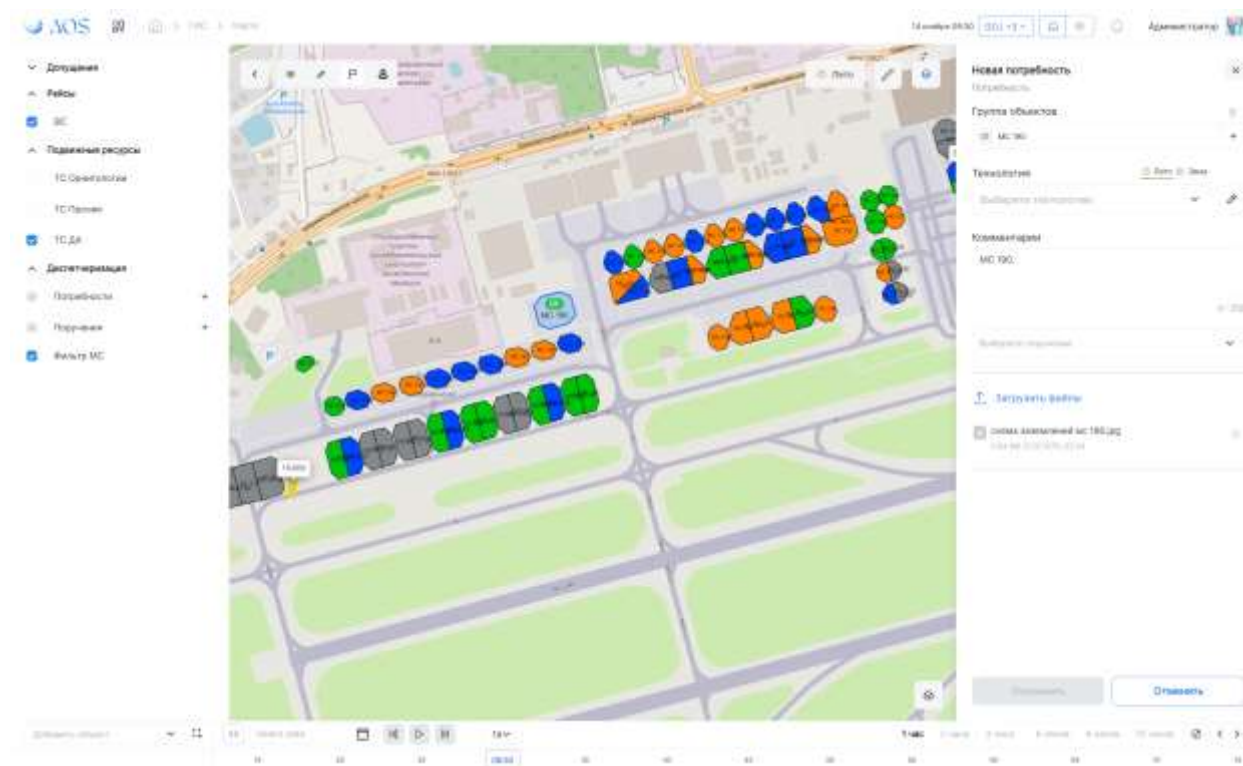


Рисунок 11 - Создание потребности

Далее от пользователя требуется выбрать вид технологии (иначе говоря вид работ), нажать на карандаш слева, и выбрать в диалоговом окне временной период выполнения задачи, а также технику (сама техника представлена в виде списка гаражных номеров, может показаться что это не лучший способ выбора техники для пользователя, одна по просьбе заказчика был выбран именно список номеров техники, так как им с ним заказчику понятнее и проще работать). Данный процесс отображен на Рисунке 12:

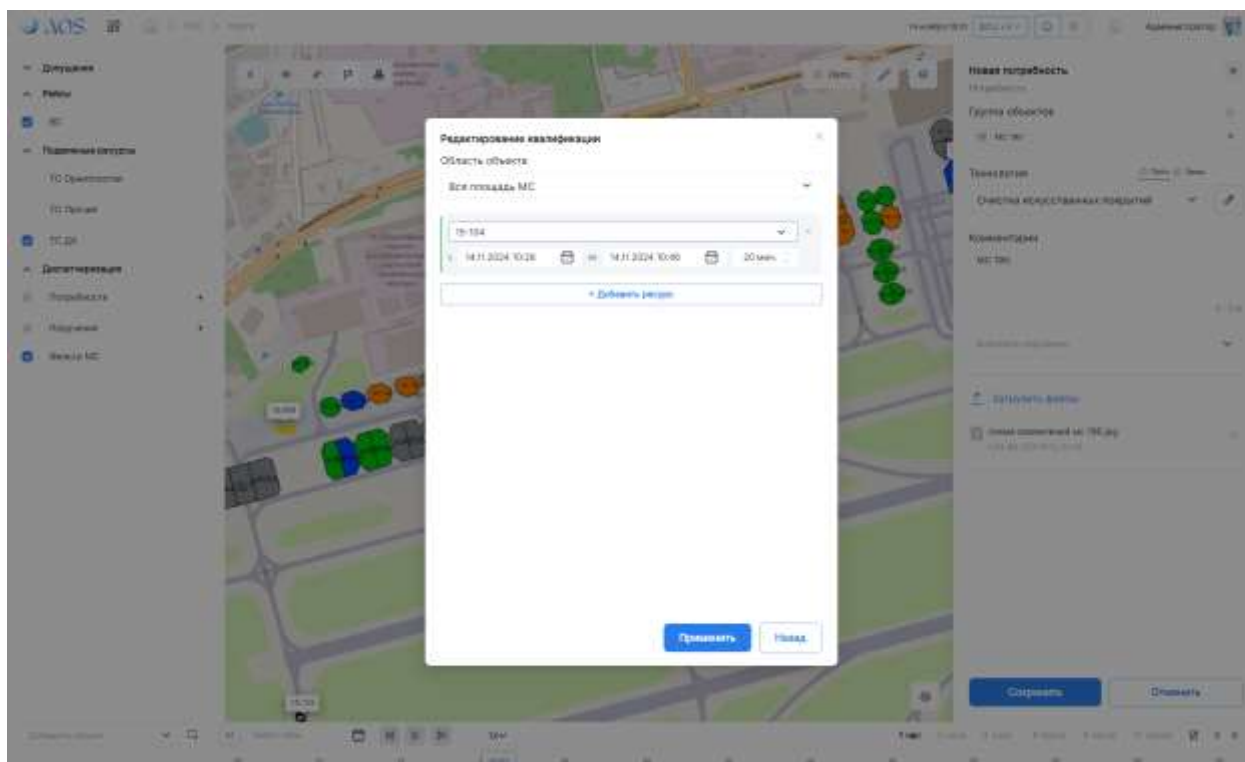


Рисунок 12 - Выбор техники

Настройка фильтров МС – вы могли заметить на Рисунках 6 и 7 покрашенные ячейки мест стоянок – это результат работы сервиса фильтрации мест стоянок. В его задачи входит постоянный пересчет правил фильтра исходя из текущих данных занятости МС и выполнения ТГО. На Рисунке 13 показано диалоговое окно настройки этих фильтров:

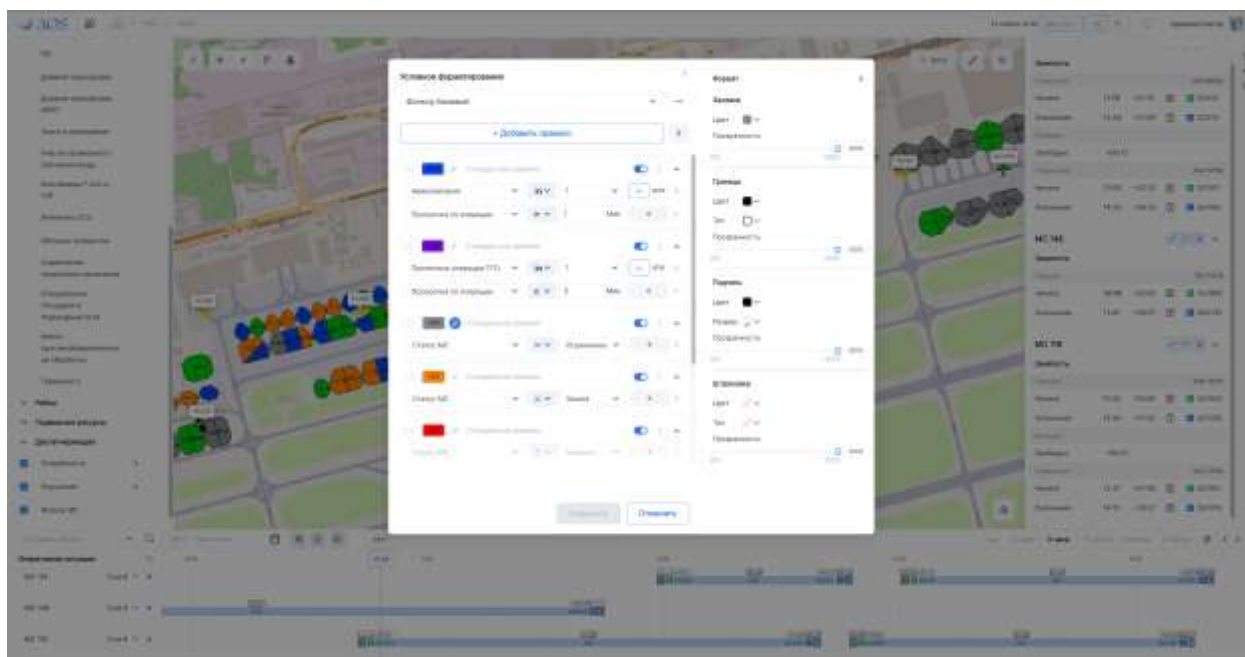


Рисунок 13 - Настройка фильтра МС

Сама система настройки фильтров выполнена очень гибко, во многом благодаря реализации в backend части алгоритма обратной постфиксной нотации, которая позволяет создавать динамические условные выражения пользователю, и обеспечивать высокую гибкость работы.

Создание и редактирования слоев – как раз работа с сервисом управления данными. Технически, данный интерфейс реализован отдельно. Это было сделано в целях снижения логической нагрузки на основной ГИС. Общий вид интерфейса представлен на Рисунке 14:



Рисунок 14 - Общий вид

На данном интерфейсе мы имеем возможность создавать новые слои, исходя из уже созданных справочников в AODB. Процесс создания слоя отображен на Рисунке 15:

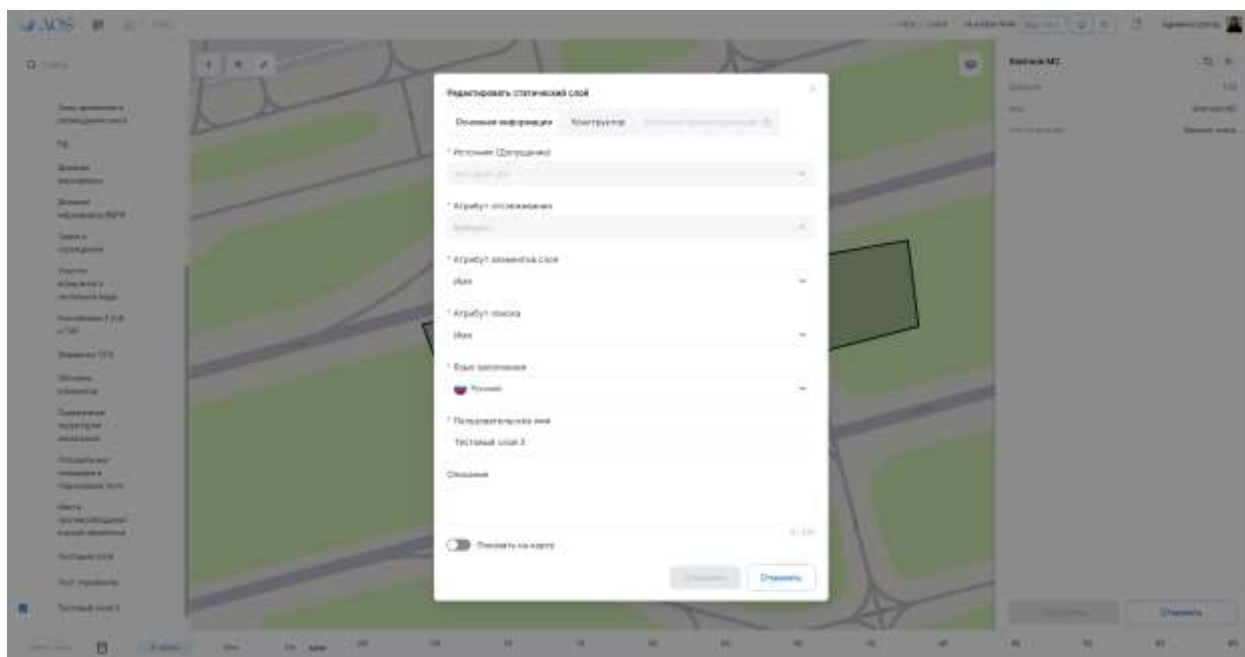


Рисунок 15 - Окно создания нового слоя

После создания слоя, в левом списке всех слоев он появляется. Для создания гео-объекта слоя, необходимо нажать на слой, затем нажать на элемент слоя, и после с помощью иконки в виде карандаша, на карте нарисовать произвольную фигуру. Этот процесс отображен на Рисунках 16 и 17:



Рисунок 16 - Процесс создания нового гео-объекта

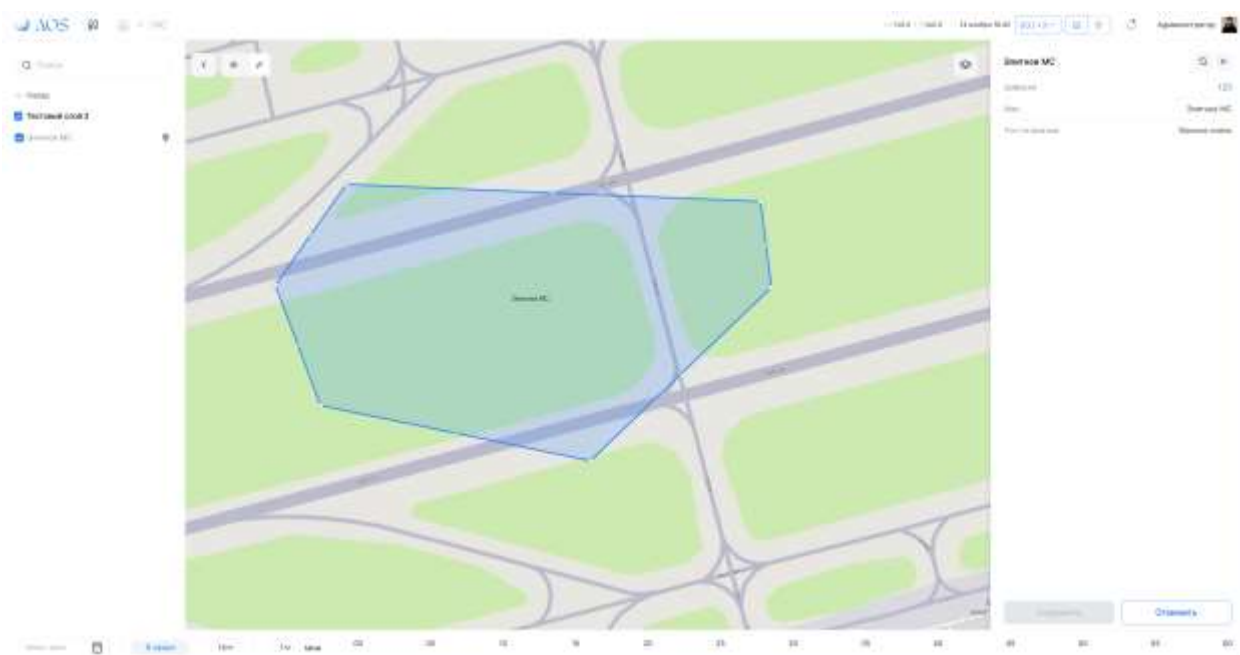


Рисунок 17 - Процесс создания нового гео-объекта

После создания объекта, пользователю требуется нажать кнопку сохранить, и объект будет добавлен в БД. Для отображения нового объекта на общем интерфейсе, в настройках слоя необходимо включить ползунок «Показать на карте».

3.2. Проверка корректности работы

3.2.1. Общие сведения

Тестирование работы приложения — это процесс проверки функциональности, стабильности и безопасности программного продукта. Вот как он может производиться:

Процесс тестирования (а точнее полного цикла тестирования и доставки ПО), по которому и осуществлялась разработка, описан ниже:

1. Планирование тестирования: - Определение целей и задач тестирования.
 - Составление тестовой документации (тест-кейсы, сценарии, чек-листы).
 - Планирование ресурсов и сроков тестирования.
2. Функциональное тестирование:
 - Проверка соответствия приложения требованиям и спецификациям.
 - Тестирование основных функций приложения (вход, регистрация, поиск, покупки и т. д.).
 - Тестирование взаимодействия с сервером, базой данных и другими сервисами.
3. Нефункциональное тестирование:
 - Тестирование производительности (нагрузка, стресс, устойчивость).
 - Тестирование безопасности (уязвимости, атаки, шифрование).
 - Тестирование совместимости (браузеры, устройства, ОС).
4. Автоматизация тестирования:
 - Использование инструментов для автоматизации тестовых сценариев.
 - Написание и запуск автоматизированных тестов.
5. Тестирование интерфейса:
 - Проверка дизайна, расположения элементов, читаемости текста.
 - Оценка удобства использования (юзабилити).
6. Регрессионное тестирование:

- Проверка того, что изменения в приложении не привели к ошибкам в уже работающих функциях.

7. Тестирование безопасности:

- Поиск уязвимостей и слабых мест в системе безопасности приложения.
- Тестирование механизмов аутентификации и авторизации.

8. Тестирование в разных средах:

- Тестирование в различных окружениях (разработка, тестирование, продуктив).
- Имитация различных условий использования (нагрузка, стабильность).

9. Сбор и анализ результатов:

- Фиксация всех найденных ошибок и проблем.
- Анализ результатов тестирования и подготовка отчёта.

10. Финальное тестирование перед релизом:

- Проверка готовности приложения к релизу.
- Устранение всех оставшихся ошибок.

11. Деплоймент и релиз:

- Развёртывание приложения на сервере.
- Релиз приложения для пользователей.

Тестирование работы приложения — это важный этап разработки, который позволяет выявить и исправить ошибки до того, как приложение будет выпущено для пользователей.

3.2.2. Тестирование backend части приложения

Теперь от теории перейдем к практике тестирования, всего у нас должно быть протестировано 6 сервисов. Для начала мы должны определиться с конкретными сценариями тестирования для каждого сервиса. Ниже перечислены основные кейсы тестов для каждого сервиса:

- `vehicle_service` – тестирование корректности обработки данных о местоположении ТС в нужную нам структуру
- `static_layer_builder` – тестирование создания и изменение слоя (CRUD-тест)
- `static_layer_connector` – тестирование корректности отдачи данных в сконвертированной структуре из AODB
- `dynamic_layer_connector` - тестирование корректности отдачи данных в сконвертированной структуре из AODB, а также корректной работы системы получения информации о занятых ТС
- `aircraft_stands_filter` – тестирование системы фильтрации MC
- `needs_service` – тестирование работы системы потребностей

На уровне программиста обычно разрабатываются unit-тесты. Интеграционные же тесты (тесты взаимодействия между сервисами) обычно проводятся специальными командами тестировщиков, так что в этой работе мы рассмотрим именно уровень unit-тестирования. Исходя из вышеописанных требований, можно составить таблицу покрытия кода. Данная метрика показывает степень покрытия кода тестами, что дает представление о полноте тестирования приложения, что представлено на Таблице 1:

Название сервиса	Покрытие	Кол-во тестов
<code>vehicle_service</code>	87%	4
<code>static_layer_builder</code>	75%	7
<code>static_layer_connector</code>	81%	9
<code>dynamic_layer_connector</code>	89%	6
<code>aircraft_stands_filter</code>	91%	12

needs_service	85%	10
---------------	-----	----

Таблица 1 - Результаты unit-тестирования

Покрытие не всегда должно соответствовать 100 процентам, по той причине, что на уровне unit-тестирования приложения, при использовании готовых библиотек взаимодействия (например, с БД), для разработчика приоритетом должно являться тестирование задач самого сервиса, т.е. непосредственное вычисление результата Б из данных А.

Подводя итог результатов тестирования backend части приложения, можно сказать, что покрытие тестами основного функционала было выполнено успешно.

3.2.3. Тестирование frontend части приложения

Frontend часть приложения так же может тестироваться с помощью unit-тестов, однако мы рассмотрим тестирования уже непосредственно со стороны конечного пользователя. Основными критериями для него конечно же является общее удобство использования, отсутствие визуальных ошибок, а также общий вид и интуитивность интерфейса. Для тестирования приложения на стороне Дирекции Аэродромной было выделено специальное производственное тестирование, в ходе которого будущие пользователи могли ознакомиться с непосредственным интерфейсом программы. В результате этого тестирования были собраны отзывы пользователей по основным компонентам тестирования, с оценкой от 1 до 10 баллов. Данные представлены на Таблице 2:

Название	Оценка	Комментарий
Удобство использования	8	Местами не совсем удобно взаимодействовать с функционалом, из-за необходимости много раз подтверждать действие
Общий вид интерфейса	10	-
Интуитивность	5	Местами интерфейс выглядит сложно
Отсутствие визуальных ошибок	8	Встречались различные ошибки обработок задач

Таблица 2 - Результаты тестирования интерфейса

Исходя из полученной обратной связи от заказчика, можно сделать следующие выводы и исправления:

- Удобство использования – первоначально, действительно, в некоторых случаях (например, создание задач) от пользователя требовалось излишнее количество действий. Как итог, где это возможно, был переделан и сокращён процесс использования того или иного функционала.
- Общий вид интерфейса – замечаний получено не было

- Интуитивность – к сожалению, учитывая специфику данного программного обеспечения, добиться полной дружелюбности интерфейса к пользователю не предоставляется возможным, так как специфика предполагает наличие знаний тех или иных процессов у пользователя, а также умение в целом пользоваться данным производственным ПО. Данное замечание исправляется только вводом обучения персонала использованию нового ПО.
- Отсутствие визуальных ошибок – местами встречались визуальные ошибки - например, проблема с погрузкой тайлов (частей изображения) у карты, наложение текста и так же некорректное отображение задач. Все найденные ошибки были исправлены, часть из них можно посмотреть в Приложении А.

Таким образом, если подводить итог тестирования frontend функционала, то можно сказать, что от пользователей в целом были получены удовлетворительные оценки использования, ошибки были устранены, неудобства исправлены.

Заключение

Подводя итоги выпускной квалификационной работы, можно сказать, что все поставленные задачи были выполнены и протестированы должным образом. За время работы была реализована геоинформационная система управления аэропортом, которая покрывает весь необходимый минимум требований Дирекции Аэродромной.

В ходе выполнения проекта были изучены современные технологии и методы геоинформационного моделирования, а также проведен анализ существующих систем управления аэропортами. На основе полученных знаний и данных была разработана архитектура системы. Кроме того, была предусмотрена возможность интеграции с существующими системами управления аэропортом, что позволяет обеспечить плавный переход на новую платформу без значительных изменений в текущих процессах. Система прошла все необходимые испытания и показала высокую эффективность и надежность в выполнении своих функций. В процессе тестирования были выявлены и устранены все выявленные недостатки, что позволило довести продукт до уровня, соответствующего требованиям заказчика.

Были выполнены следующие задачи:

- Разработана общая концепция ПО
- Проведен анализ предметной области, проведено общее исследование темы
- Выбран и обоснован стек ПО
- Выбрана и обоснована архитектура ПО
- Разработано приложение;
- Реализованы основные функциональные требования к программному обеспечению;
- Проведено тестирование приложения;

Однако, так же остаются и перспективы совершенствования и развития продукта, например – развитие системы ГИС в рамках задач Дирекции

Управления полетами (углубление функционала постановки поручений), а также Орнитологической службы.

Таким образом, можно с уверенностью сказать, что цель выпускной квалификационной работы достигнута, и разработанная геоинформационная система управления аэропортом является готовым к внедрению решением, способным повысить эффективность и безопасность работы аэропорта.

Список использованных источников

Законодательные и нормативные акты

1. Указ Президента РФ от 1 мая 2022 г. N 250 "О дополнительных мерах по обеспечению информационной безопасности Российской Федерации" от 1 мая 2022 г.
2. ГОСТ 7.82 – 2001. Библиографическая запись. Библиографическое описание электронных ресурсов. Общие требования и правила составления. – М.: ИПК Издательство стандартов, 2001. – 21 с.
3. ГОСТ 7.1. – 2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – М.: ИПК Издательство стандартов, 2004. – 169 с.
4. ГОСТ 7.32 – 2001. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. – М.: ИПК Издательство стандартов, 2001. – 21 с.

Учебная и научная литература

1. Лутц М. Изучаем Python. – Москва, Символ-Полус, 2011. – 1280 с.
2. Лутц М. Программирование на Python. Том 1. – Москва, Символ-Полус, 2011. – 992 с.
3. Дронов В. Django 2.1. Практика создания веб-сайтов на Python - Москва: BHV, 2019. – 672 с.
4. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. — М.: Питер, 2018. — 352 с.
5. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер. — М.: Вильямс, 2006. — 544 с.
6. Лафоре, Р. Объектно-ориентированное программирование в C++ / Р. Лафоре. — СПб.: Питер, 2013. — 928 с.
7. Брукс, Ф. Мифический человеко-месяц, или Как создаются программные системы / Ф. Брукс. — СПб.: Символ-Плюс, 2010. — 304 с.

8. Леффингуэлл, Д. Принципы работы с требованиями к программному обеспечению. Унифицированный процесс разработки / Д. Леффингуэлл, Д. Уидриг. — М.: Вильямс, 2002. — 448 с.
9. Орлик, С. Ю. Введение в архитектуру программного обеспечения / С. Ю. Орлик. — М.: ИНТУИТ, 2016. — 286 с.
10. Эмблер, С. Свидание с абстракцией / С. Эмблер. — М.: ДМК Пресс, 2021. — 224 с.
11. Гамма, Э. Приёмы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес. — СПб.: Питер, 2019. — 400 с.
12. Канер, С. Тестирование программного обеспечения. Фундаментальные концепции менеджмента бизнес-приложений / С. Канер, Дж. Фолк, Е. Кнг. — М.: Вильямс, 2001. — 544 с.
13. Ховард, М. Защищённый код / М. Ховард, Д. Лебланк, Д. Виега. — М.: Русская редакция, 2004. — 704 с.
14. Буч, Г. Объектно-ориентированный анализ и проектирование с примерами приложений / Г. Буч. — М.: Бином, 1998. — 560 с.
15. Липовских, В. В. Основы алгоритмизации и программирования / В. В. Липовских. — М.: ИНТУИТ, 2016. — 326 с.
16. Фридл, Дж. Регулярные выражения / Дж. Фридл. — СПб.: Питер, 2002. — 352 с.
17. Буч, Г. Объектно-ориентированный анализ и проектирование / Г. Буч, Д. Рамбо, А. Джекобсон. — М.: Бином, 19
18. Макконнелл, С. Совершенный код. Практическое руководство по разработке программного обеспечения / С. Макконнелл. — М.: Русская редакция, 2004. — 896 с.

Интернет-ресурсы

1. ООО "Код Открытия" / [Электронный ресурс] // : [сайт]. — URL: <https://cd-it.ru/> (дата обращения: 12.10.2024). — Текст: электронный.

2. Международный аэропорт Шереметьево имени А. С. Пушкина / [Электронный ресурс] // : [сайт]. — URL: <https://www.svo.aero/ru/main> (дата обращения: 12.10.2024). — Текст: электронный.
3. Официальная документация Python / [Электронный ресурс] // : [сайт]. — URL: <https://docs.python.org/> (дата обращения: 12.10.2024). — Текст: электронный.
4. Learn Python / [Электронный ресурс] // : [сайт]. — URL: <https://learnpython.org/> (дата обращения: 12.10.2024). — Текст: электронный.
5. Python.org / [Электронный ресурс] // : [сайт]. — URL: <https://python.org/> (дата обращения: 12.10.2024). — Текст: электронный.
6. Stack Overflow / [Электронный ресурс] // : [сайт]. — URL: <https://stackoverflow.com/> (дата обращения: 12.10.2024). — Текст: электронный.
7. Real Python / [Электронный ресурс] // : [сайт]. — URL: <https://realpython.com/> (дата обращения: 12.10.2024). — Текст: электронный.
8. Python Central / [Электронный ресурс] // : [сайт]. — URL: <https://python-central.org> (дата обращения: 12.10.2024). — Текст: электронный.
9. PyCon / [Электронный ресурс] // : [сайт]. — URL: <https://www.pycon.org/> (дата обращения: 12.10.2024). — Текст: электронный.
10. PyPI / [Электронный ресурс] // : [сайт]. — URL: <https://pypi.org/> (дата обращения: 12.10.2024). — Текст: электронный.
11. Python Forum / [Электронный ресурс] // : [сайт]. — URL: <https://forums.python.org> (дата обращения: 12.10.2024). — Текст: электронный.
12. Django / [Электронный ресурс] // : [сайт]. — URL: <https://www.djangoproject.com/> (дата обращения: 12.10.2024). — Текст: электронный.

- 13.Flask / [Электронный ресурс] // : [сайт]. — URL: <https://flask.palletsprojects.com/> (дата обращения: 12.10.2024). — Текст: электронный.
- 14.Django REST Framework / [Электронный ресурс] // : [сайт]. — URL: <https://djangorestframework.org/> (дата обращения: 12.10.2024). — Текст: электронный.
- 15.Requests / [Электронный ресурс] // : [сайт]. — URL: <https://requests-cookbook.readthedocs.io> (дата обращения: 12.10.2024). — Текст: электронный.
- 16.Pytest / [Электронный ресурс] // : [сайт]. — URL: <https://pytest.org/> (дата обращения: 12.10.2024). — Текст: электронный.
- 17.FastAPI / [Электронный ресурс] // : [сайт]. — URL: <https://fastapi.tiangolo.com/> (дата обращения: 12.10.2024). — Текст: электронный.
- 18.Pydantic / [Электронный ресурс] // : [сайт]. — URL: <https://docs.pydantic.dev/latest/> (дата обращения: 12.10.2024). — Текст: электронный.
- 19.PostgreSQL / [Электронный ресурс] // : [сайт]. — URL: <https://www.postgresql.org/> (дата обращения: 12.10.2024). — Текст: электронный.
- 20.MongoDB / [Электронный ресурс] // : [сайт]. — URL: <https://www.mongodb.com> (дата обращения: 12.10.2024). — Текст: электронный.
- 21.Redis / [Электронный ресурс] // : [сайт]. — URL: <https://redis.io/> (дата обращения: 12.10.2024). — Текст: электронный.
- 22.Kafka / [Электронный ресурс] // : [сайт]. — URL: <https://kafka.apache.org> (дата обращения: 12.10.2024). — Текст: электронный.

Приложение А

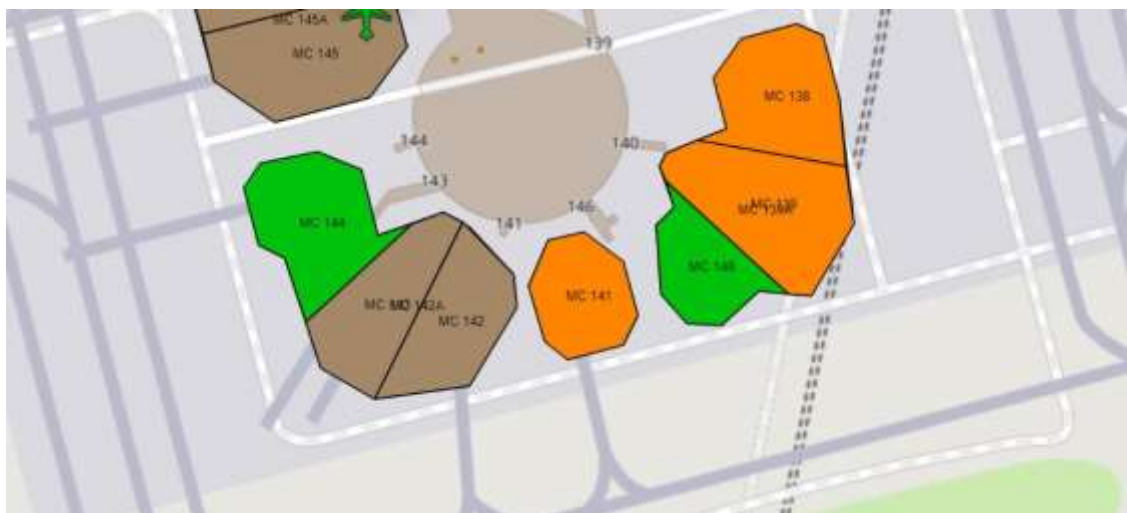


Рисунок 18 - Визуальная ошибка отображения надписей мест стоянок

Очистка искусственных покры...     

Потребность

Статус В работе

Поручение

Группа объектов
[MC 209](#)

Технология
Очистка искусственных покрытий
Область объекта: Вся площадь MC
Время начала: 31.10.2024 12:10 (+00:01)
Время окончания: 31.10.2024 12:30

Задача В обработке

 Загрузить файлы

Задача Завершена

Исполнитель:    15-110
Время начала: 31.10.2024 12:10 (+00:01)
Время окончания: 31.10.2024 12:30 (-00:09)

Рисунок 19 - Неверное отображение задачи